

M0單晶片－1

高秀娟

Oct. of 2021

Install Keil MDK

Google : Keil mdk



keil mdk



全部

圖片

影片

購物

新聞

更多

工具

約有 560,000 項結果 (搜尋時間 : 0.45 秒)

<https://www2.keil.com> › mdk5 ▾ [翻譯這個網頁](#)

MDK Version 5 - Keil

Keil® MDK is the most comprehensive software development solution for Arm®-based microcontrollers and includes all components that you need to create, build, ...

[Compare MDK Editions](#) · [MDK-Lite Edition](#) · [MDK-Professional Edition](#) · [µVision IDE](#)

<https://www.keil.com> ▾ [翻譯這個網頁](#)

Keil Embedded Development Tools for Arm, Cortex-M, Cortex ...

Keil MDK is the complete software development environment for a range of Arm Cortex-M based microcontroller devices. MDK includes the µVision IDE and ...

<https://www2.keil.com/mdk5>



[Home](#) / [MDK](#)

[Products](#)

[Download](#)

[Events](#)

[Support](#)

[Go](#)

 Learning Platform

MDK Microcontroller Development Kit

Keil® MDK is the most comprehensive software development solution for Arm®-based microcontrollers and includes all components that you need to create, build, and debug embedded applications.

[Download MDK](#)

Update: MDK v5.34 contains Arm Compiler 6.16 and further enhancements.

 **Quick Links**

- [Getting Started](#)
- [Online Manuals](#)
- [Compare Editions](#)
- [Middleware](#)
- [Learn](#)

 **Software Packs**

- [Device List](#)
- [Evaluation Boards](#)
- [MDK v4 Legacy Support](#)
- [Third-Party Software Packs](#)
- [Public Software Packs](#)

Write correct gmail

MDK-ARM

MDK-ARM Version 5.36

Version 5.36

Complete the following form to download the Keil software development tools.

Enter Your Contact Information Below

First Name:

Last Name:

E-mail:

Company:

Job Title:

Country/Region:

Phone:

☐ Send me e-mail when there is a new update.

NOTICE:

If you select this check box, you **will** receive an e-mail message from Keil whenever a new update is available. If you don't wish to receive an e-mail notification, don't check this box.

Which device are you using?
(eg, STM32)

Arm will process your information in accordance with the Evaluation section of our [Privacy Policy](#).

☐ Please keep me updated on products, services and other relevant offerings from Arm. You can change your mind and unsubscribe at any time.

Submit

Reset

Install Software Packs

Pack Installer - C:\Users\julia\AppData\Local\Arm\Packs

File Packs Window Help

Device: STMicroelectronics - STM32F072RBTx

Devices

Boards

Search:

Device

Summary

All Devices	8741 Devices
ABOV Semiconductor	30 Devices
Active-Semi	17 Devices
Ambiq Micro	10 Devices
Amicom	5 Devices
Analog Devices	15 Devices
APEXMIC	25 Devices
ARM	58 Devices
AutoChips	53 Devices
BrainChip	1 Device
Cmsemicon	32 Devices
Cypress	931 Devices
Dialog Semiconductor	15 Devices
ELAN	1 Device
EtaCompute	3 Devices
FMD	9 Devices
FMSH	3 Devices
Geehy	54 Devices
GinaDevice	703 Devices

Packs

Examples

Pack	Action	Description
Device Specific	2 Packs	STM32F072RBTx selected
Keil::STM32F0xx_DFP	Up to date	STMicroelectronics STM32F0 Series Device Support, Drivers and Examples
Keil::STM32NUCLEO_B...	Up to date	STMicroelectronics Nucleo Boards Support and Examples
Generic	58 Packs	
Alibaba::AliOSThings	Install	AliOS Things software pack
Arm-Packs::PKCS11	Install	OASIS PKCS #11 Cryptographic Token Interface
Arm-Packs::Unity	Install	Unit Testing for C (especially Embedded Software)
ARM::AMP	Install	Software components for inter processor communication (Asymmetric M
ARM::CMSIS	Up to date	CMSIS (Common Microcontroller Software Interface Standard)
ARM::CMSIS-Driver	Up to date	CMSIS Drivers for external devices
ARM::CMSIS-Driver_Va...	Install	CMSIS-Driver Validation
ARM::CMSIS-FreeRTOS	Install	Bundle of FreeRTOS for Cortex-M and Cortex-A
ARM::CMSIS-RTOS_Vali...	Install	CMSIS-RTOS Validation
ARM::ethos-u-core-dri...	Install	Device Driver for the Arm(R) Ethos(TM)-U NPU.
ARM::mbedClient	Install	ARM mbed Client for Cortex-M devices
ARM::mbedCrypto	Install	ARM mbed Cryptographic library
ARM::mbedTLS	Install+	ARM mbed Cryptographic and SSL/TLS library
ARM::minar	Install	mbed OS Scheduler for Cortex-M devices
ARM::PSA	Install	PSA (Platform Security Architecture)
ARM::TFM	Install+	Trusted Firmware-M (TF-M) reference implementation of Arm's Platform

File Packs Window Help

Device: STMicroelectronics - STM32F072RB

Devices Boards

Search:

Device	Summary
SONIX	68 Devices
STMicroelectronics	1857 Devices
STBlueNRG Series	1 Device
STBlueNRG-1 Series	1 Device
STBlueNRG-2 Series	1 Device
STBlueNRG-LP Series	2 Devices
STM32F0 Series	111 Devices
STM32F030	7 Devices
STM32F031	10 Devices
STM32F038	5 Devices
STM32F042	13 Devices
STM32F048	3 Devices
STM32F051	17 Devices
STM32F058	4 Devices
STM32F070	4 Devices
STM32F071	10 Devices
STM32F072	13 Devices
STM32F072C8	2 Devices
STM32F072CB	3 Devices
STM32F072R8	1 Device
STM32F072RB	3 Devices
STM32F072RBHx	ARM Cortex-M0, 48 MHz, 16 kB RAM, 128 kB ROM
STM32F072RBLx	ARM Cortex-M0, 48 MHz, 16 kB RAM, 128 kB ROM
STM32F072RBTx	ARM Cortex-M0, 48 MHz, 16 kB RAM, 128 kB ROM
STM32F072V8	2 Devices
STM32F072VB	2 Devices
STM32F078	7 Devices

Packs Examples

Pack	Action	Description
Device Specific	2 Packs	STM32F072RB selected
Keil::STM32F0xx_DFP	Up to date	STMicroelectronics STM32F0 Series Device Support, Drivers and Examples
2.1.0 (2020-05-04)	Remove	STMicroelectronics STM32F0 Series Device Support, Drivers and Examples
Previous		Keil::STM32F0xx_DFP - Previous Pack Versions
Keil::STM32NUCLEO_BSP	Up to date	STMicroelectronics Nucleo Boards Support and Examples
1.8.0 (2021-07-13)	Remove	STMicroelectronics Nucleo Boards Support and Examples
Previous		Keil::STM32NUCLEO_BSP - Previous Pack Versions
Generic	58 Packs	
Alibaba::AliOSThings	Install	AliOS Things software pack
Arm-Packs::PKCS11	Install	OASIS PKCS #11 Cryptographic Token Interface
Arm-Packs::Unity	Install	Unit Testing for C (especially Embedded Software)
ARM::AMP	Install	Software components for inter processor communication (Asymmetric Multi Proce
ARM::CMSIS	Up to date	CMSIS (Common Microcontroller Software Interface Standard)
ARM::CMSIS-Driver	Up to date	CMSIS Drivers for external devices
ARM::CMSIS-Driver_Validation	Install	CMSIS-Driver Validation
ARM::CMSIS-FreeRTOS	Install	Bundle of FreeRTOS for Cortex-M and Cortex-A
ARM::CMSIS-RTOS_Validation	Install	CMSIS-RTOS Validation
ARM::ethos-u-core-driver	Install	Device Driver for the Arm(R) Ethos(TM)-U NPU.
ARM::mbedClient	Install	ARM mbed Client for Cortex-M devices
ARM::mbedCrypto	Install	ARM mbed Cryptographic library
ARM::mbedTLS	Install+	ARM mbed Cryptographic and SSL/TLS library
ARM::miniar	Install	mbed OS Scheduler for Cortex-M devices
ARM::PSA	Install	PSA (Platform Security Architecture)
ARM::TFM	Install+	Trusted Firmware-M (TF-M) reference implementation of Arm's Platform Security A
ARM::TFM-Test	Install+	Trusted Firmware-M (TF-M) Tests
ARM::V2M-MPS3_SSE_300_TFM-PF	Install+	ARM V2M-MPS3 TF-M Platform Support pack.
ASN::Filter_Designer	Install	Intuitive graphical FIR/IIR digital filter designer for CMSIS-DSP projects

Output

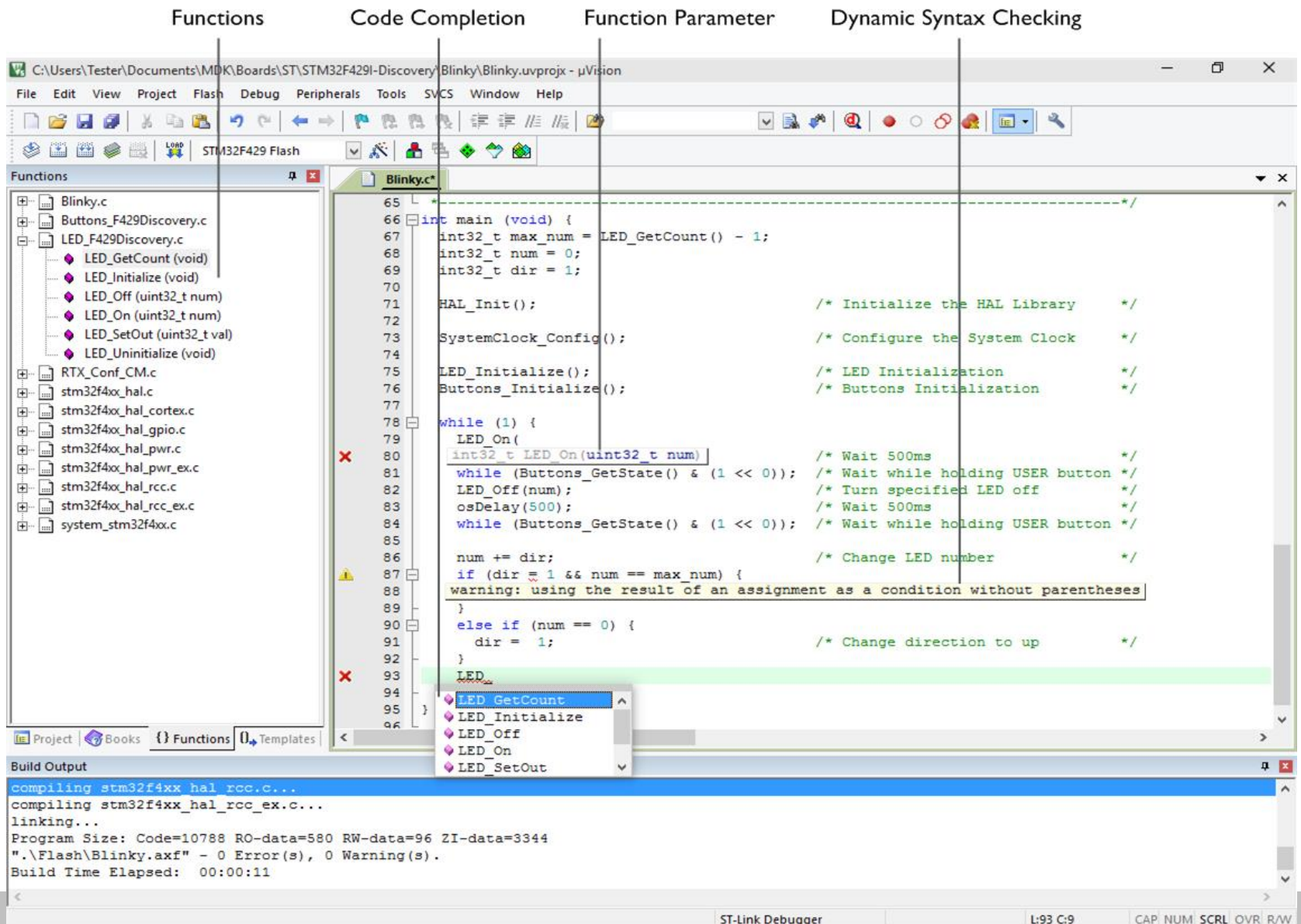
Refresh Pack descriptions

Check for updates

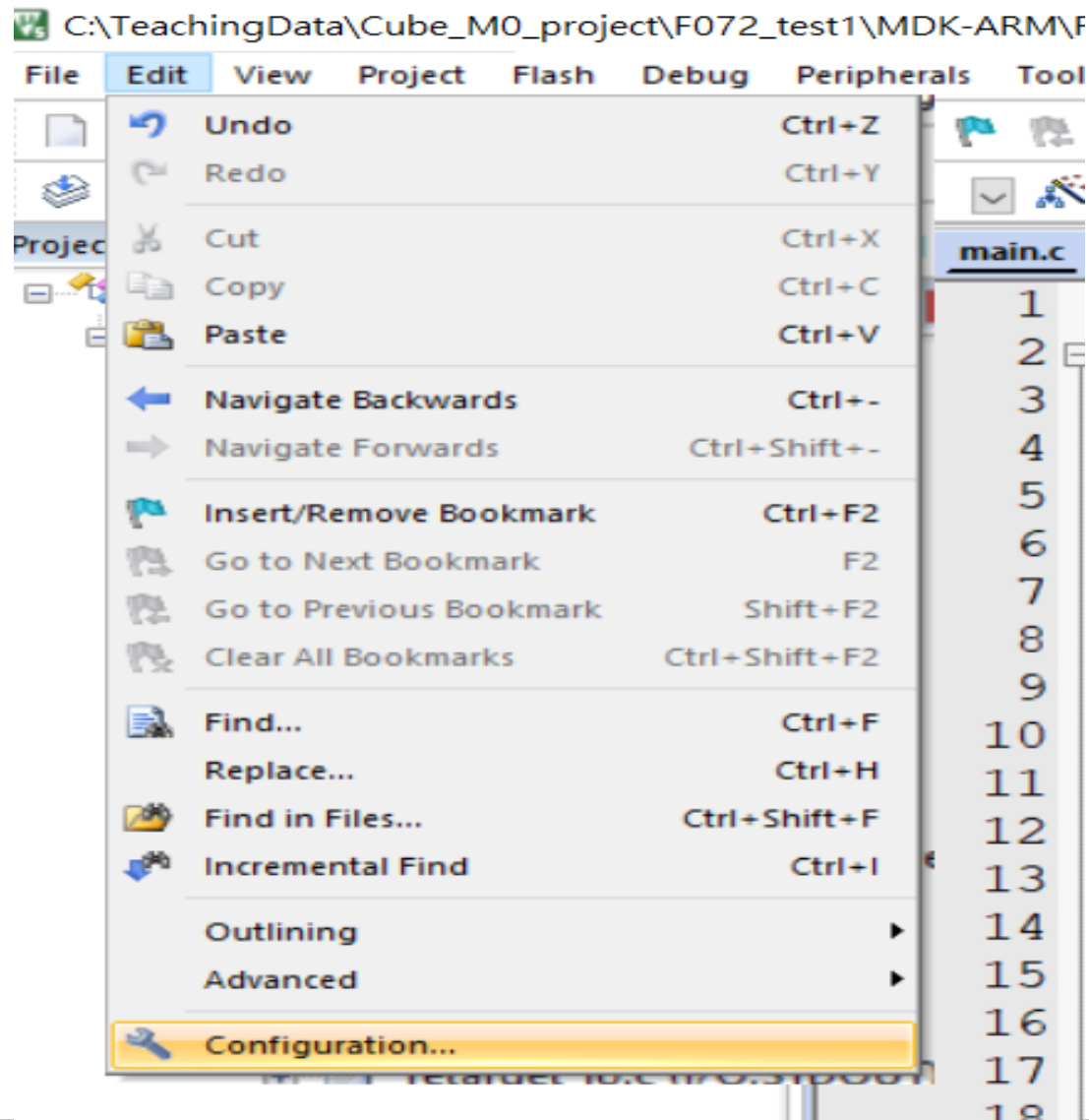
Completed requested actions

ONLINE

µVision IDE

Functions	Code Completion	Function Parameter	Dynamic Syntax Checking
 Functions - Blinky.c - Buttons_F429Discovery.c - LED_F429Discovery.c - LED_GetCount (void) - LED_Initialize (void) - LED_Off (uint32_t num) - LED_On (uint32_t num) - LED_SetOut (uint32_t val) - LED_Uninitialize (void) - RTX_Conf_CM.c - stm32f4xx_hal.c - stm32f4xx_hal_cortex.c - stm32f4xx_hal_gpio.c - stm32f4xx_hal_pwr.c - stm32f4xx_hal_pwr_ex.c - stm32f4xx_hal_rcc.c - stm32f4xx_hal_rcc_ex.c - system_stm32f4xx.c Code Completion <pre>65 66 int main (void) { 67 int32_t max_num = LED_GetCount() - 1; 68 int32_t num = 0; 69 int32_t dir = 1; 70 71 HAL_Init(); 72 /* Initialize the HAL Library */ 73 SystemClock_Config(); 74 /* Configure the System Clock */ 75 LED_Initialize(); 76 Buttons_Initialize(); 77 /* LED Initialization */ 78 /* Buttons Initialization */ 79 while (1) { 80 LED_On(81 int32_t LED_On(uint32_t num) 82 while (Buttons_GetState() & (1 << 0)); 83 /* Wait 500ms */ 84 LED_Off(num); 85 /* Turn specified LED off */ 86 osDelay(500); 87 /* Wait 500ms */ 88 while (Buttons_GetState() & (1 << 0)); 89 /* Wait while holding USER button */ 90 num += dir; 91 /* Change LED number */ 92 if (dir == 1 && num == max_num) { 93 warning: using the result of an assignment as a condition without parentheses 94 } 95 else if (num == 0) { 96 dir = 1; 97 /* Change direction to up */ 98 } 99 LED 100 } 101 } 102 }</pre> Function Parameter - LED_GetCount - LED_Initialize - LED_Off - LED_On - LED_SetOut Dynamic Syntax Checking warning: using the result of an assignment as a condition without parentheses Build Output <pre>compiling stm32f4xx_hal_rcc.c... compiling stm32f4xx_hal_rcc_ex.c... linking... Program Size: Code=10788 RO-data=580 RW-data=96 ZI-data=3344 ".\Flash\Blinky.axf" - 0 Error(s), 0 Warning(s). Build Time Elapsed: 00:00:11</pre>			

Configuration



ReBuild & Program the code

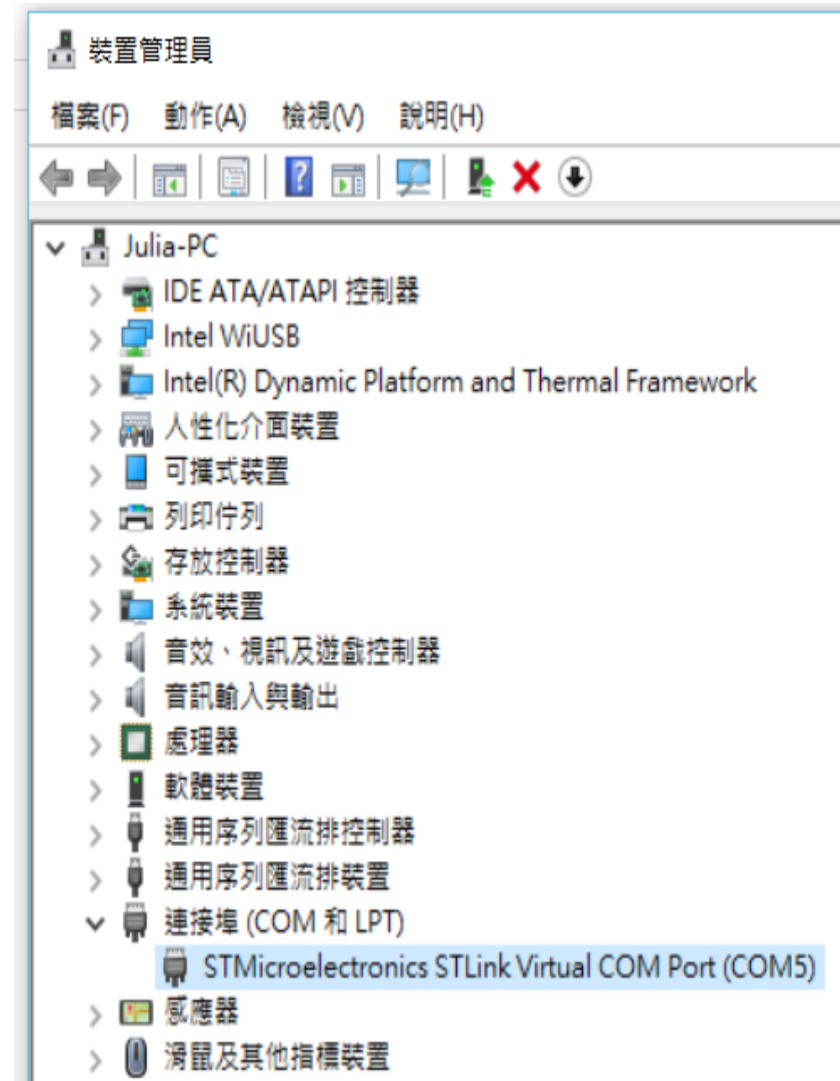
- ❖ Open Blinky project from STM32_NUCLEO_BSP 目錄.
- ❖  ReBuild all Target files.
- ❖  Program the application into Flash memory. From the toolbar, choose Download.
 The Build Output window shows messages about the download progress.

Build Output

```
*** Using Compiler 'V5.06 update 2 (build 183)', folder: 'E:\Keil_v5\ARM\ARMCC\Bin'
Rebuild target 'STM32F072 Flash'
compiling Buttons_NUCLEO_F072RB.c...
compiling Blinky.c...
compiling stdout_USART.c...
assembling startup_stm32f072xb.s...
compiling retarget_io.c...
compiling LED_NUCLEO_F072RB.c...
compiling system_stm32f0xx.c...
linking...
Program Size: Code=1848 RO-data=244 RW-data=16 ZI-data=1024
".\Flash\Blinky.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:04
```

ST-LINK/V2-1 installation

- ❖ All the STM32 Nucleo boards include an ST-LINK/V2-1 embedded debug tool. This tool requires a dedicated USB driver, which can be found at www.st.com . Unzip **stsw-link009 USB driver** 執行dpinst_amd64.exe.
- ❖ The ST-LINK/V2-1 embeds a firmware upgrade mechanism in the board. Before starting debug user has to run **stsw-link007 upgrade firmware** to board through the USB port. 執行/Windows/ST-LinkUpgrade.exe.



Install STM CubeMx

**STM32 configuration and
initialization C code generation**

Google : stm cubemx



stm cubemx



全部

圖片

影片

新聞

購物

更多

工具

約有 61,200 項結果 (搜尋時間 : 0.45 秒)

包含符合以下字詞的搜尋結果 : **stm32 cubemx**

僅搜尋以下字詞 : **stm cubemx**

<https://www.st.com> › development-tools › stm32cubemx

STM32CubeMX - STM32Cube initialization code generator

STM32CubeMX is a graphical tool that allows a very easy configuration of **STM32** microcontrollers and microprocessors, as well as the generation of the ...

<https://www.st.com> › resource › user_manual › dm0...

UM1718 – User manual - STMicroelectronics

2021年6月1日 — **STM32CubeMX** for **STM32** configuration and initialization C code generation. Introduction. **STM32CubeMX** is a graphical tool for **STM32** products.

<https://www.st.com/en/development-tools/stm32cubemx.html>

[ations](#)  [Solutions](#)  [Tools & Software](#)  [About ST](#)

Development Tools > STM32 Software Development Tools > STM32 Configurators and Code Generators > STM32CubeMX >

STM32CubeMX

ACTIVE

STM32Cube initialization code generator

[Get Software](#) [!\[\]\(abdaa36b47fa525142932abbc2512e5b_img.jpg\) Download databrief](#)

[Overview](#) [Documentation](#) [Tools & Software](#)

Get Software

Get Software

	Part Number	General Description	Latest version	Download	All versions
+	STM32CubeMX-Lin	STM32Cube init code generator for Linux	6.3.0	Get latest	Select version ▼
+	STM32CubeMX-Mac	STM32Cube init code generator for macOS	6.3.0	Get latest	Select version ▼
+	STM32CubeMX-Win	STM32Cube init code generator for Windows	6.3.0	Get latest	Select version ▼

Agree License agreement



License Agreement

ACCEPT

Please indicate your acceptance or NON-acceptance by selecting "I ACCEPT" or "I DO NOT ACCEPT" as indicated below in the media.

BY INSTALLING COPYING, DOWNLOADING, ACCESSING OR OTHERWISE USING THIS SOFTWARE PACKAGE OR ANY PART THEREOF (AND THE RELATED DOCUMENTATION) FROM STMICROELECTRONICS INTERNATIONAL N.V, SWISS BRANCH AND/OR ITS AFFILIATED COMPANIES (STMICROELECTRONICS), THE RECIPIENT, ON BEHALF OF HIMSELF OR HERSELF, OR ON BEHALF OF ANY ENTITY BY WHICH SUCH RECIPIENT IS EMPLOYED AND/OR ENGAGED AGREES TO BE BOUND BY THIS SOFTWARE PACKAGE LICENSE AGREEMENT.

Under STMicroelectronics' intellectual property rights and subject to applicable licensing terms for any third-party software incorporated in this software package and applicable Open Source Terms (as defined here below), the redistribution, reproduction and use in source and binary forms of the software package or any part thereof, with or without modification, are permitted provided that the following conditions are met:

1. Redistribution of source code (modified or not) must retain any copyright notice, this list of conditions and the following disclaimer.

Register and Login



Get Software

If you have an account on my.st.com, login and download the software without any further validation steps.

Login/Register

If you don't want to login now, you can download the software by simply providing your name and e-mail address in the form below and validating it.

This allows us to stay in contact and inform you about updates of this software.

For subsequent downloads this step will not be required for most of our software.

First Name:

Last Name:

E-mail address:

Please review our [Privacy Statement](#) that describes how we process your profile information and how to assert your personal data protection rights

☐ Please keep me informed about future updates for this software or new software in the same category

Download

執行 CubeMX



File

Window

Help



Home

Existing Projects

Recent Opened Projects

F072_test1.ioc

MX

Last modified date : 21/07/2021 11:00:16

STCubeGenerated.ioc

MX

Last modified date : 07/10/2021 19:34:52

Test_1.ioc

MX

Last modified date : 28/09/2021 13:36:25

Other Projects



New Project

I need to :

Start My project from MCU

ACCESS TO MCU SELECTOR

Start My project from ST Board

ACCESS TO BOARD SELECTOR

Start My project from Example

ACCESS TO EXAMPLE SELECTOR

Manage software installations

Check for STM32CubeMX and embe...

CHECK FOR UPDATES

Install or remove embedded software ...

INSTALL / REMOVE



Project manager setup

Pinout & Configuration	Clock Configuration
Project	Project Settings Project Name F072_test1 Project Location C:\TeachingData\Cube_M0_project Application Structure Advanced ☐ Do not generate the main() Toolchain Folder Location C:\TeachingData\Cube_M0_project\F072_test1\ Toolchain / IDE MDK-ARM Min Version V5.32 ☐ Generate Under Root
Code Generator	Linker Settings Minimum Heap Size 0x200 Minimum Stack Size 0x400
Advanced Settings	Thread-safe Settings Cortex-M0NS ☐ Enable multi-threaded support Thread-safe Locking Strategy Default - Mapping suitable strategy depending on RTOS selection.

MDK專案環境設定

設定開發環境

Options for Target 'STM32F072 Flash'

Device Target Output Listing User C/C++ Asm Linker Debug Utilities

STMicroelectronics STM32F072RB

Xtal (MHz): 48.0

Operating system: None

System Viewer File: STM32F0x2.svd

☐ Use Custom File

Code Generation

ARM Compiler: Use default compiler version

☐ Use Cross-Module Optimization

☒ Use MicroLIB ☐ Big Endian

Read/Only Memory Areas

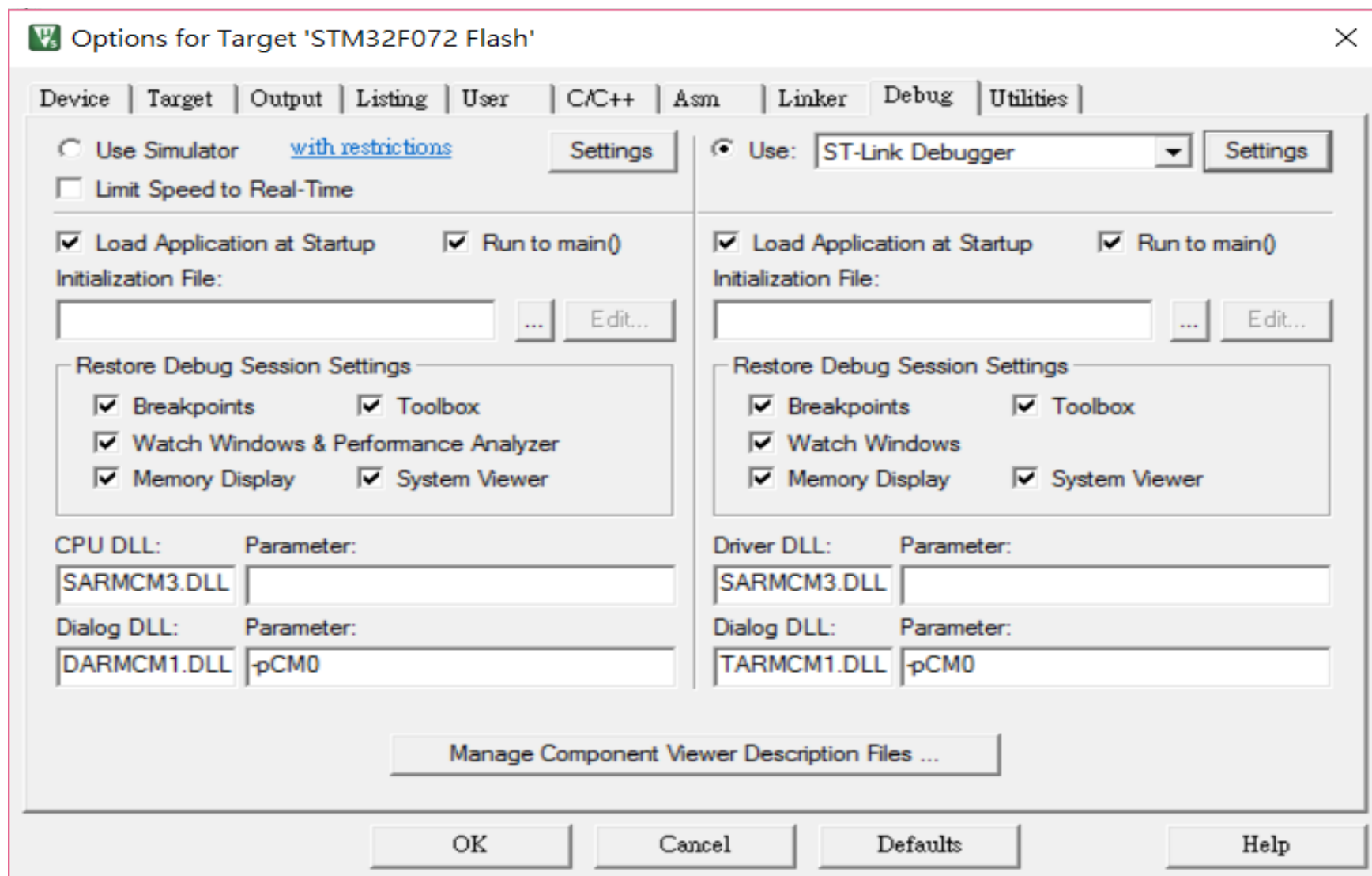
	default	off-chip	Start	Size	Startup
☐		ROM1:			☐
☐		ROM2:			☐
☐		ROM3:			☐
		on-chip			
☒		IROM1:	0x8000000	0x20000	☒
☐		IROM2:			☐

Read/Write Memory Areas

	default	off-chip	Start	Size	NoInit
☐		RAM1:			☐
☐		RAM2:			☐
☐		RAM3:			☐
		on-chip			
☒		IRAM1:	0x20000000	0x4000	☐
☐		IRAM2:			☐

OK Cancel Defaults Help

Debug: 選擇和設定 debug tool



設定 Flash Download

- ❖ The section RAM for Algorithm defines the address space where programming algorithms are loaded and executed, usually located in on-chip RAM.

Cortex-M Target Driver Setup

Debug | Trace | **Flash Download**

Download Function

 ☐ Erase Full Chip ☒ Program

☒ Erase Sectors ☒ Verify

☐ Do not Erase ☒ Reset and Run

RAM for Algorithm

Start: Size:

Programming Algorithm

Description	Device Size	Device Type	Address Range
STM32F0xx 128kB Flash	128k	On-chip Flash	08000000H - 0801FFFFH

Start: Size:

Run-Time Environment

設定 Run-Time Environment (RTE)

Manage Run-Time Environment

Software Component	Sel.	Variant	Version	Description
Board Support		NUCLEO-F072RB	1.1.0	STMicroelectronics NUCLEO-F072RB Development Board
CMSIS				Cortex Microcontroller Software Interface Components
CORE	☒		5.5.0	CMSIS-CORE for Cortex-M, SC000, SC300, ARMv8-M, ARMv8.1-M
DSP	☐	Source	1.9.0-dev	CMSIS-DSP Library for Cortex-M, SC000, and SC300
NN Lib	☐		3.0.0	CMSIS-NN Neural Network Library
RTOS (API)			1.0.0	CMSIS-RTOS API for Cortex-M, SC000, and SC300
RTOS2 (API)			2.1.3	CMSIS-RTOS API for Cortex-M, SC000, and SC300
CMSIS Driver				Unified Device Drivers compliant to CMSIS-Driver Specifications
Compiler		ARM Compiler	1.6.0	Compiler Extensions for ARM Compiler 5 and ARM Compiler 6
Event Recorder	☐	DAP	1.4.0	Event Recording and Component Viewer via Debug Access Port (DAP)
I/O				Retarget Input/Output
File	☐	File System	1.2.0	Use retargeting together with the File System component
STDERR	☐	Breakpoint	1.2.0	Stop program execution at a breakpoint when using STDERR
STDIN	☐	Breakpoint	1.2.0	Stop program execution at a breakpoint when using STDIN
STDOUT	☒	User	1.2.0	Redirect STDOUT to a user defined output target (USART, Graphics Display or other)
TTY	☐	Breakpoint	1.2.0	Stop program execution at a breakpoint when using TTY
Device		STM32CubeMX	1.0.0	Peripheral, Clock and Pin configuration are managed by STM32CubeMX
File System		MDK-Plus	6.14.1	File Access on various storage devices
Graphics		MDK-Plus	6.16.3	User Interface on graphical LCD displays
Graphics Display				
Network		MDK-Plus	7.15.0	IPv4 Networking using Ethernet or Serial protocols
USB		MDK-Plus	6.15.0	USB Communication with various device classes

STM32 MCU series for IoT

- The leading platform of the IoT world



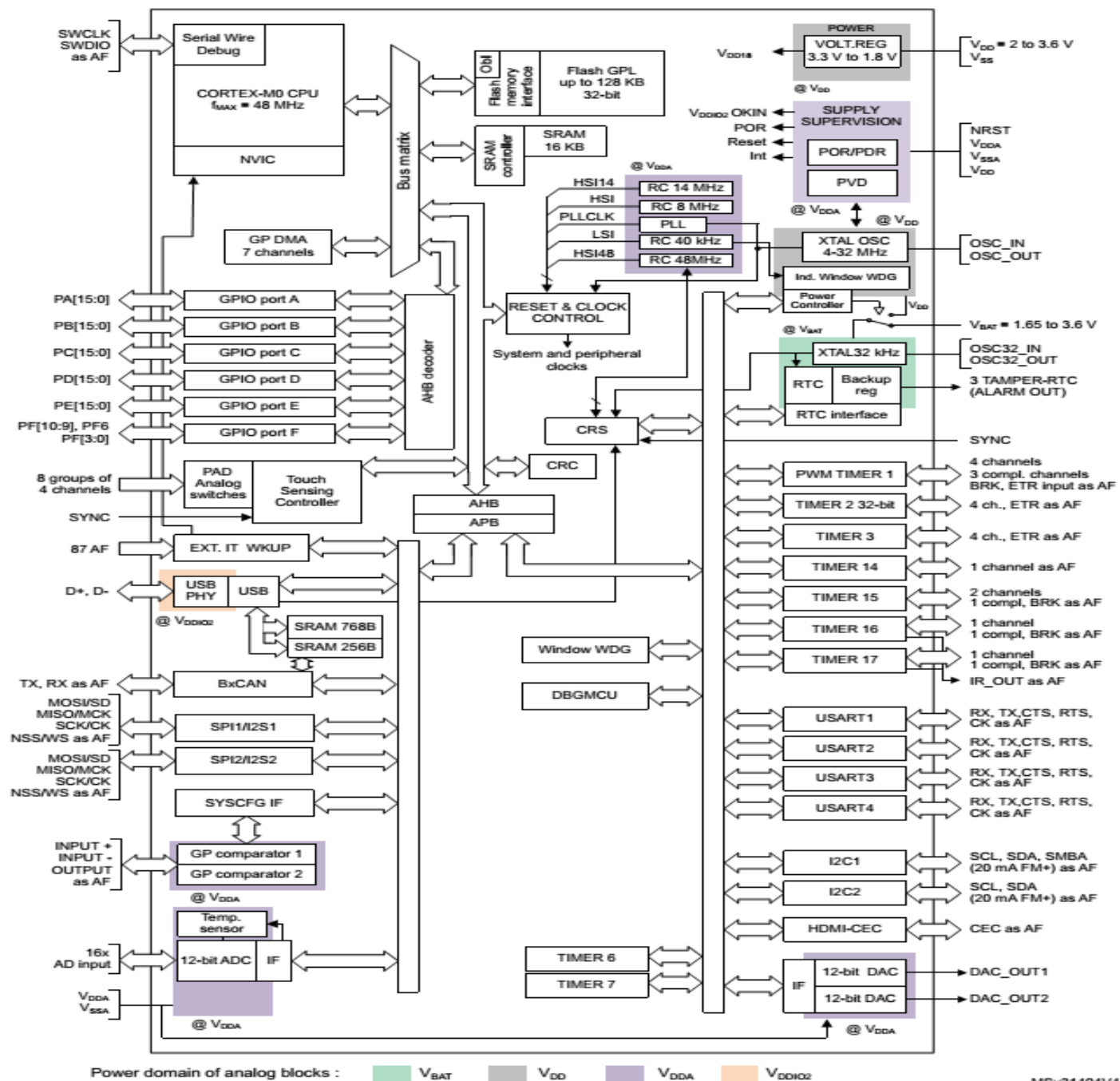
- A valued and broad ecosystem helping innovation for a smarter future



STM32F072 Features

STM32F072

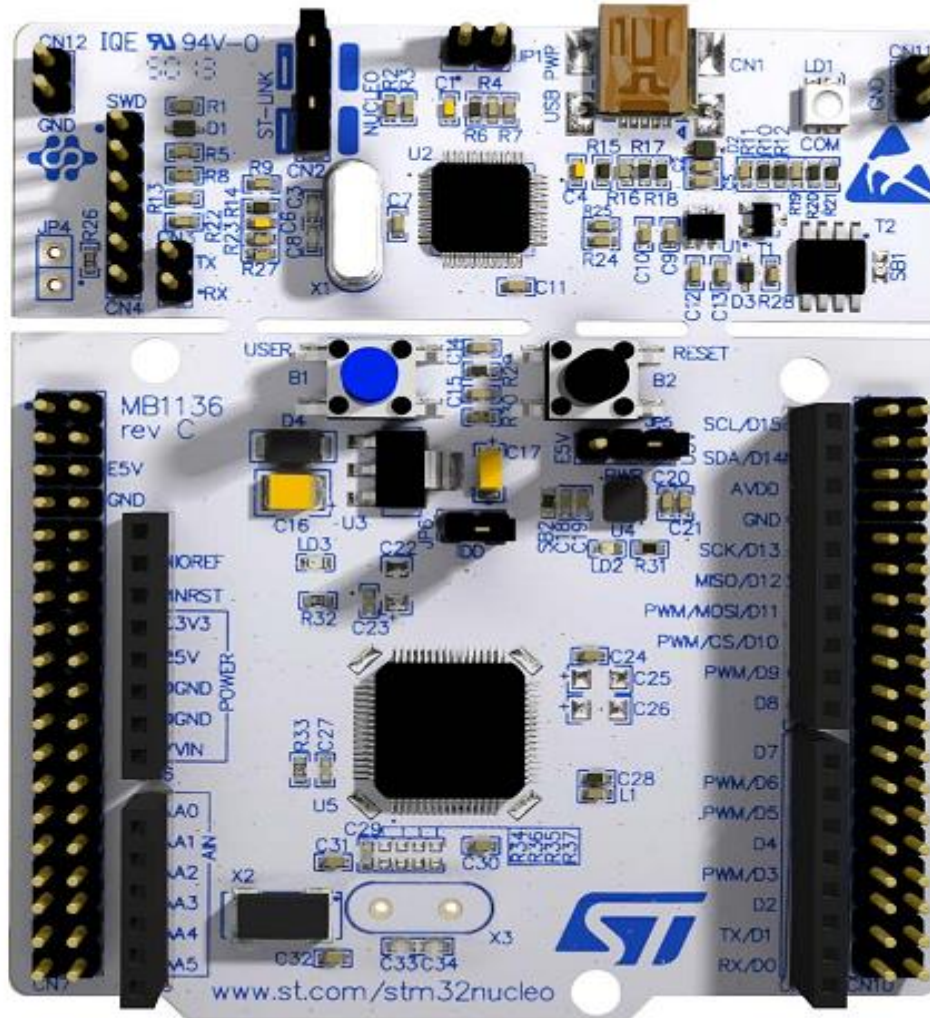
Block Diagram



STM32F072 key features

- ❖ Core: ARM 32-bit Cortex-M0 CPU, frequency up to 48 MHz
- ❖ Memories : 128 Kbytes of Flash memory , 16 Kbytes of SRAM
- ❖ Digital and I/O supply: VDD= 2.0 V to 3.6 V
- ❖ Up to 87 fast I/Os
- ❖ 12 timers with PWM
- ❖ Calendar RTC with alarm
- ❖ Two I2C interfaces supporting Fast Mode Plus (1 Mbit/s)
- ❖ Four USARTs
- ❖ Two SPIs (18 Mbit/s)
- ❖ CAN interface
- ❖ USB 2.0 full-speed interface
- ❖ One 12-bit, 1.0 μ s ADC (up to 16 channels)
- ❖ One 12-bit D/A converter (with 2 channels)
- ❖ Up to 24 capacitive sensing channels for touch key
- ❖ HDMI CEC wakeup on header reception

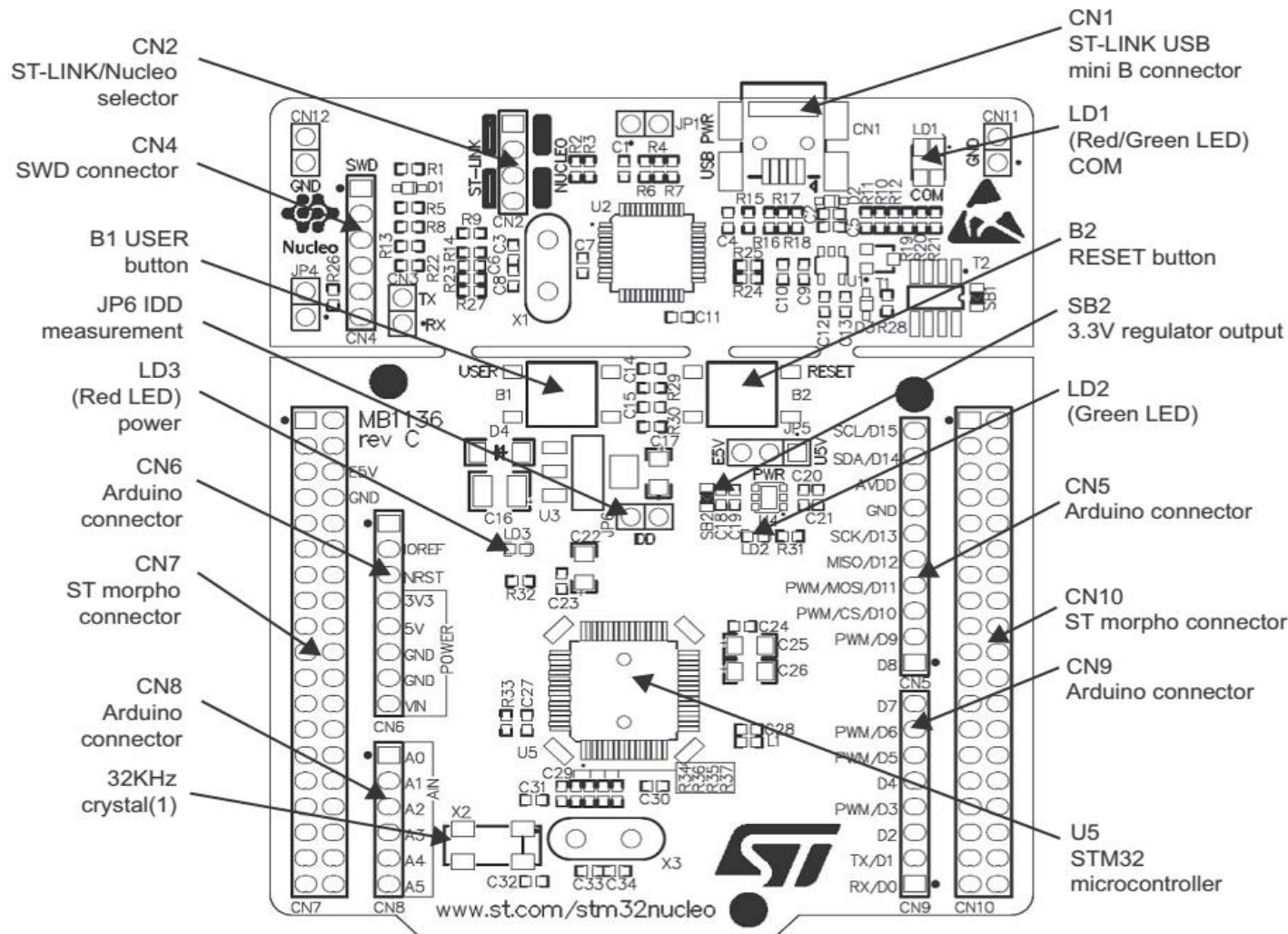
STM32 Nucleo-64 development board



- On-board ST-LINK/V2-1 chip for debugger/programmer
- STM32F072 MCU with LQFP64 package

www.st.com/stm32nucleo

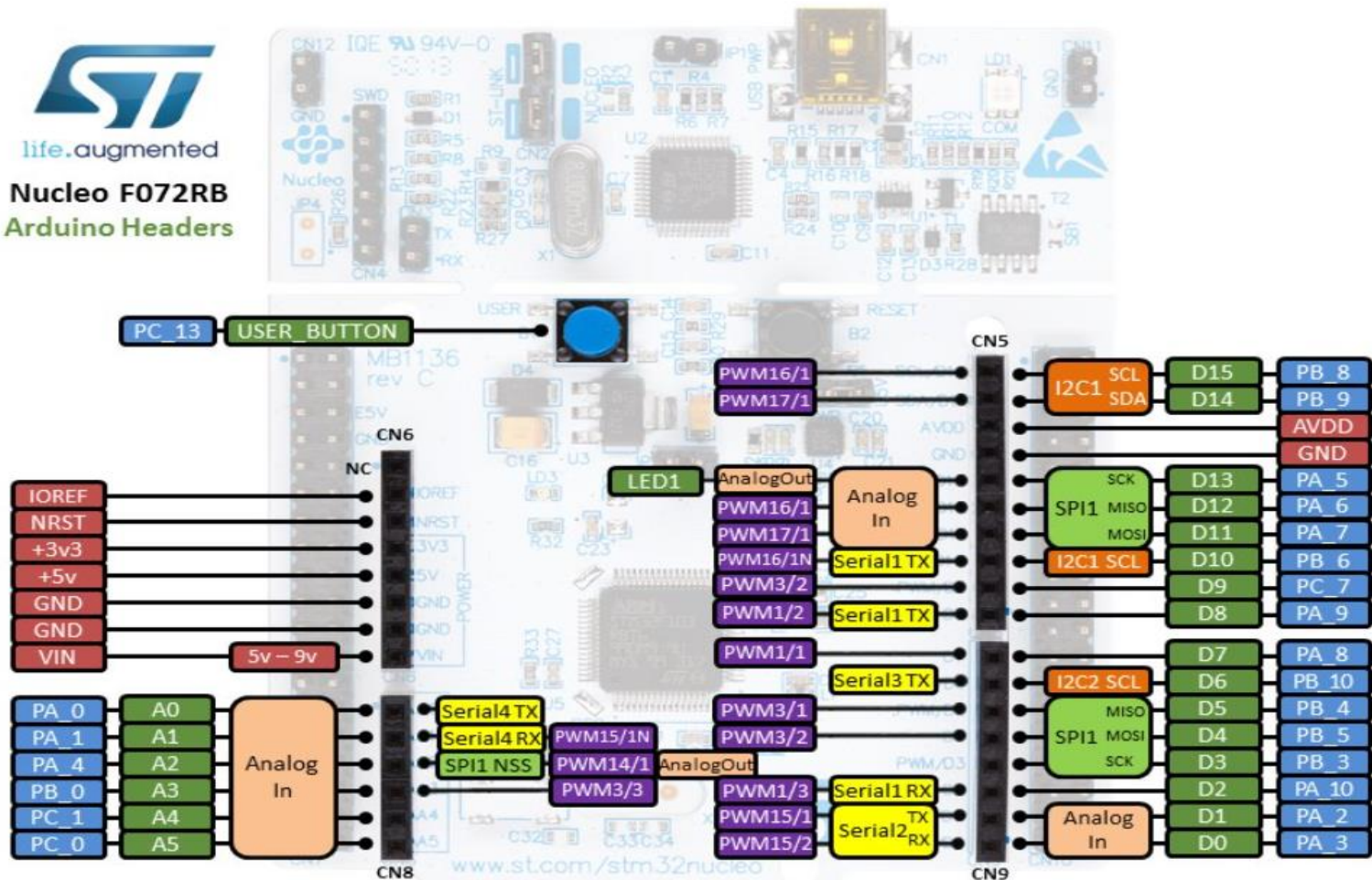
Board Top Layout



- ❖ Button B1 (left button) for USER. Button B2 (right button) for RESET
- ❖ Connect the STM32 Nucleo board to a PC through USB connector CN1.
- ❖ The red LED LD3 for Power on , LD1 for USB port.
- ❖ The jumper position: JP5 for Power source(U5V), CN2 for NUCLEO selected.

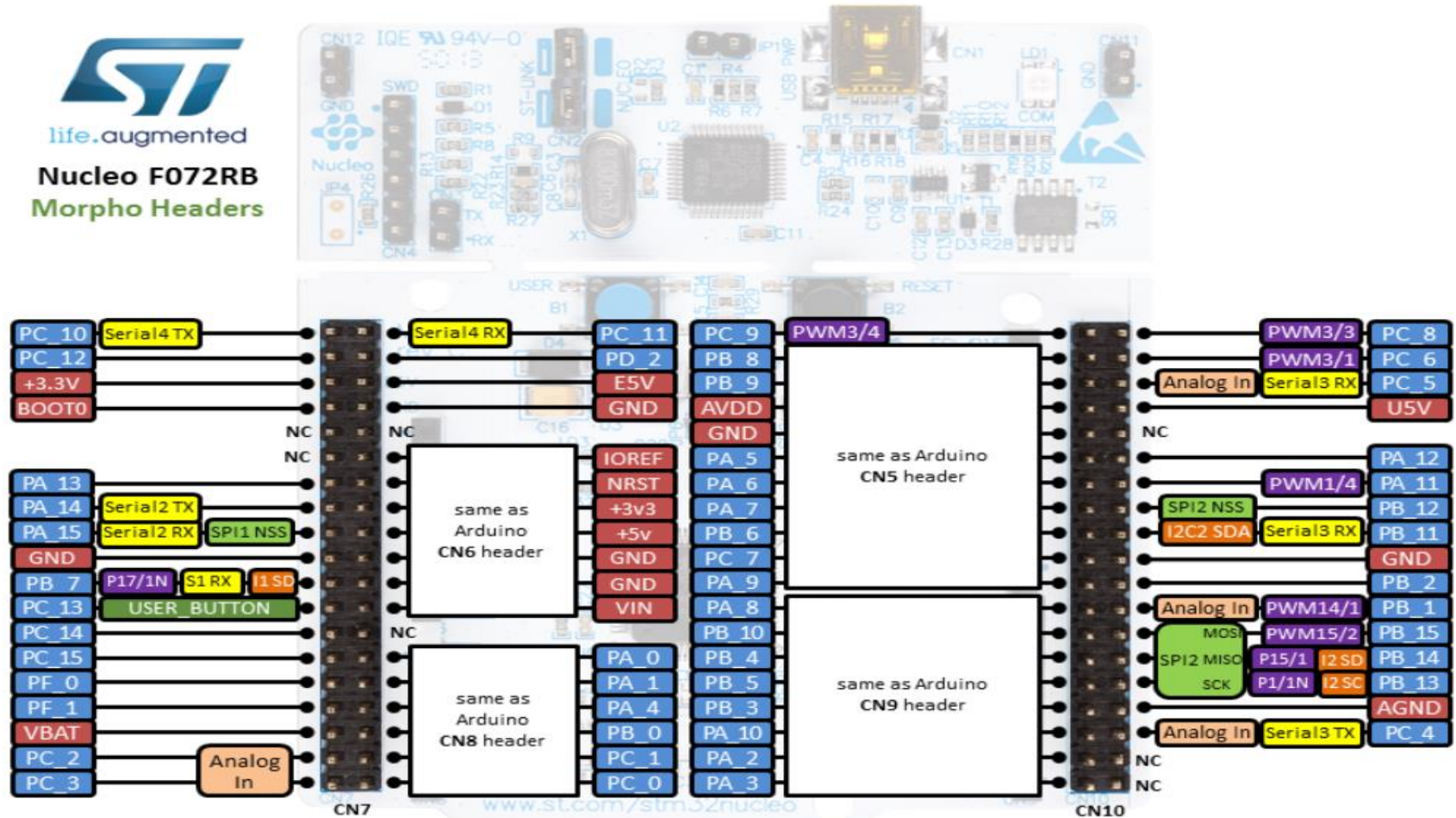
Arduino-compatible headers : CN6、CN8、CN5、CN9


life.augmented
Nucleo F072RB
Arduino Headers



PS: 綠底白字為Arduino的接腳名稱, 藍底白字代表實際其對應的MCU腳位. 其他顏色的字為該腳位的多功能選項

Morpho headers: link with all MCU I/O pins

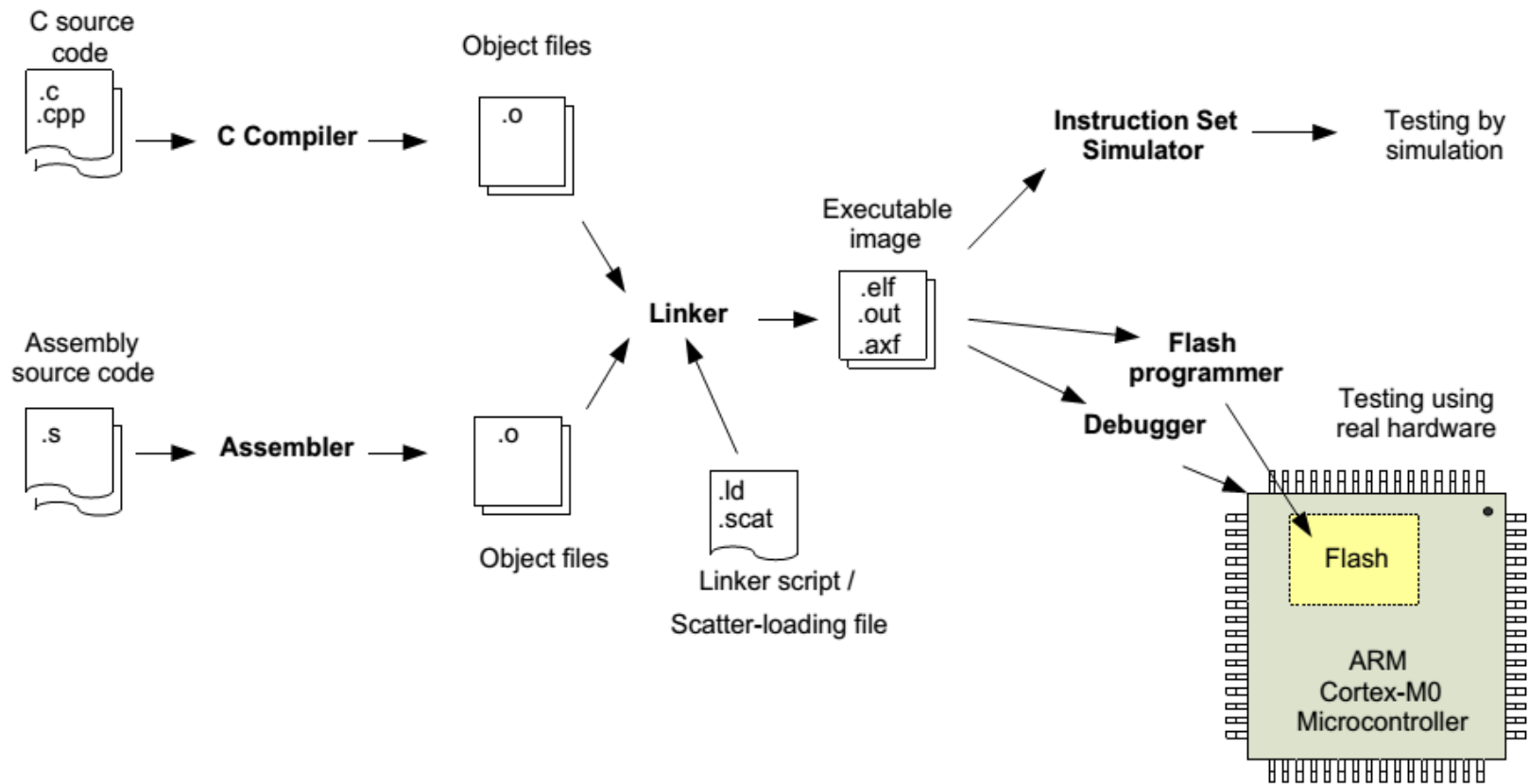


PS:藍底白字代表實際其對應的MCU腳位.其他顏色的字為該腳位的多功能選項

MCU Program Structure

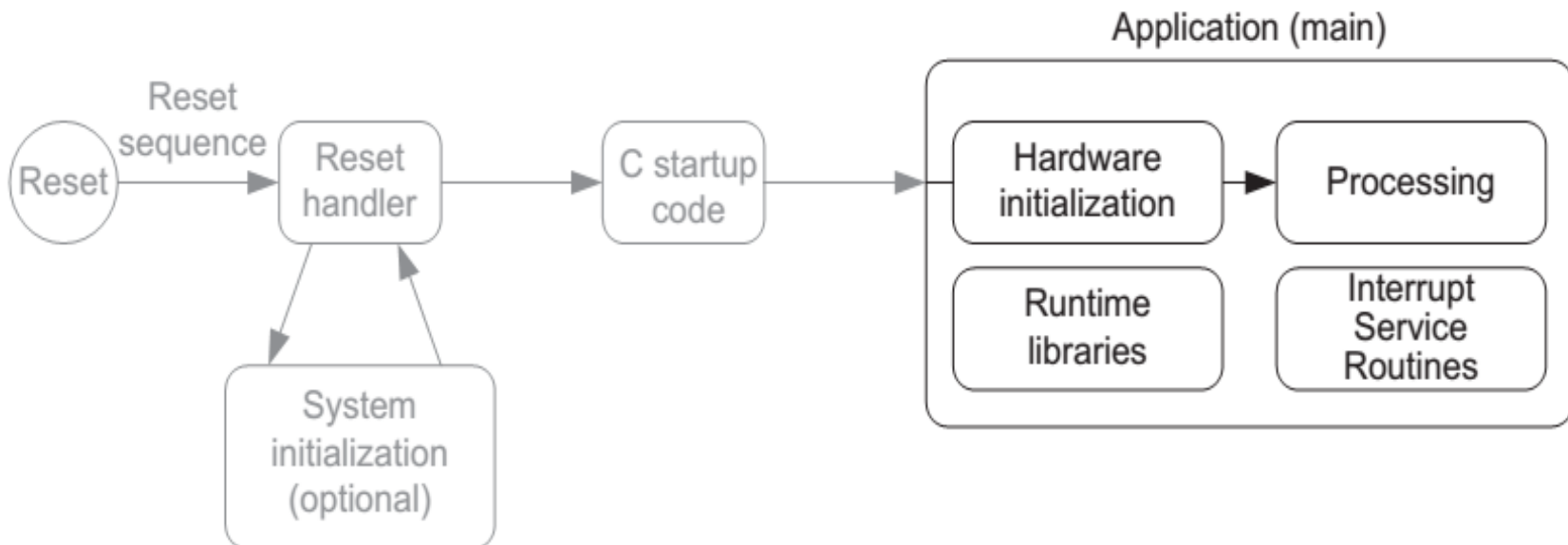
M0 Development tool chains

- ❖ 由於現代程式都相當複雜，常將程式設計工作區分為多個C/C++程式。將這些程式使用編譯器(Compiler)編譯為物件檔之後，必須使用連結器(Linker)將這些物件檔一併連結變成單一機器語言程式。接著必須將產生的目的碼寫入至適當的記憶體位置，以便由CPU執行。

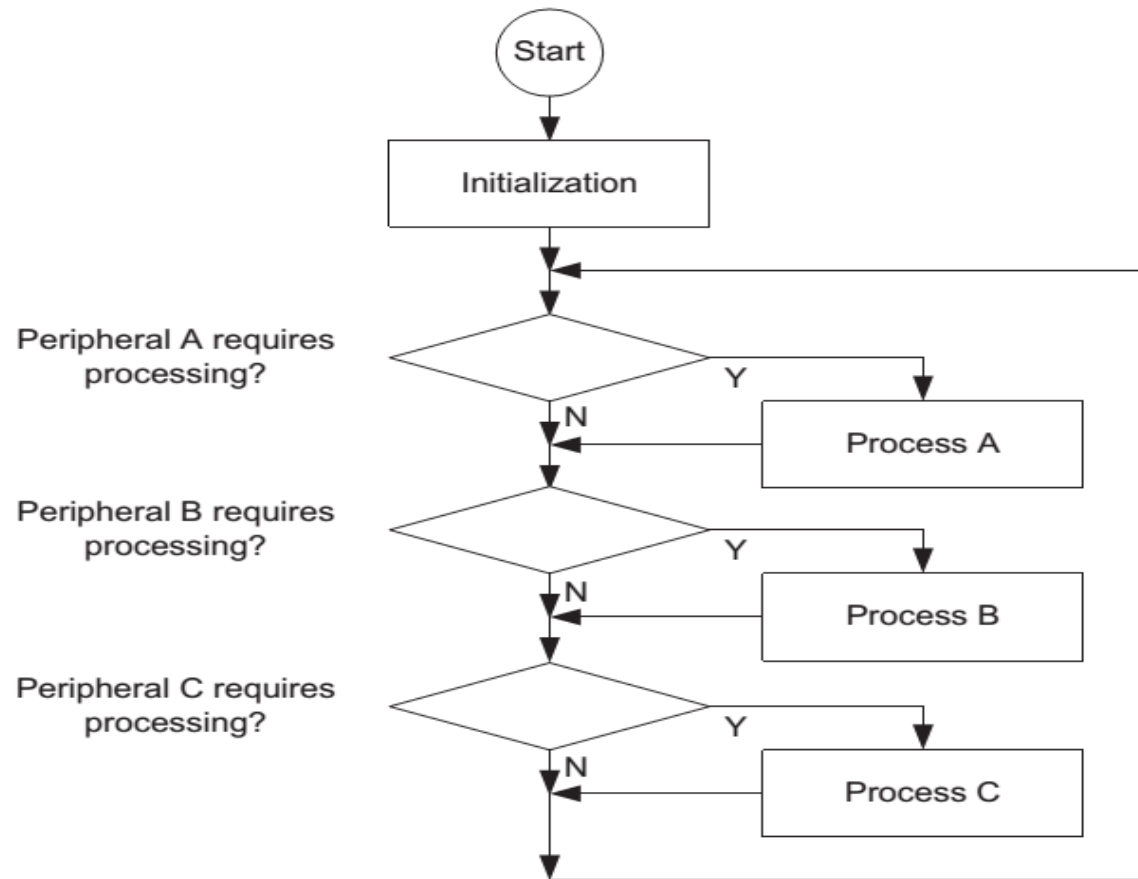


MCU Program Structure

- ❖ The application program often contains the following elements:
 - Initialization of hardware (e.g., clock, PLL, peripherals : GPIO, USART, ...)
 - The processing part of the application
 - Interrupt service routines
 - Use library. The compiler/linker will include the required library functions into the compiled program image.



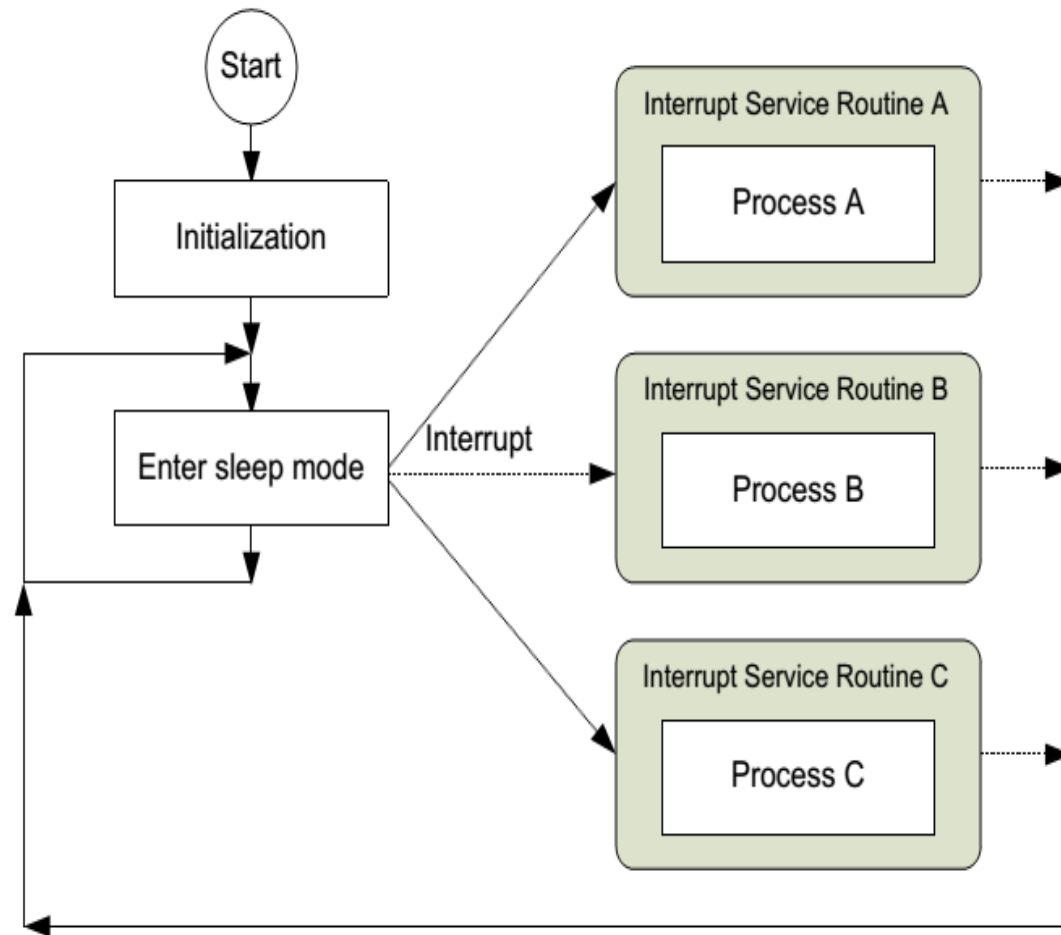
Polling



❖ Polling

- For simple applications, polling (also called super loop) is easy to set up and works well.
- when the application gets complicated and demands higher performance, polling is not suitable.

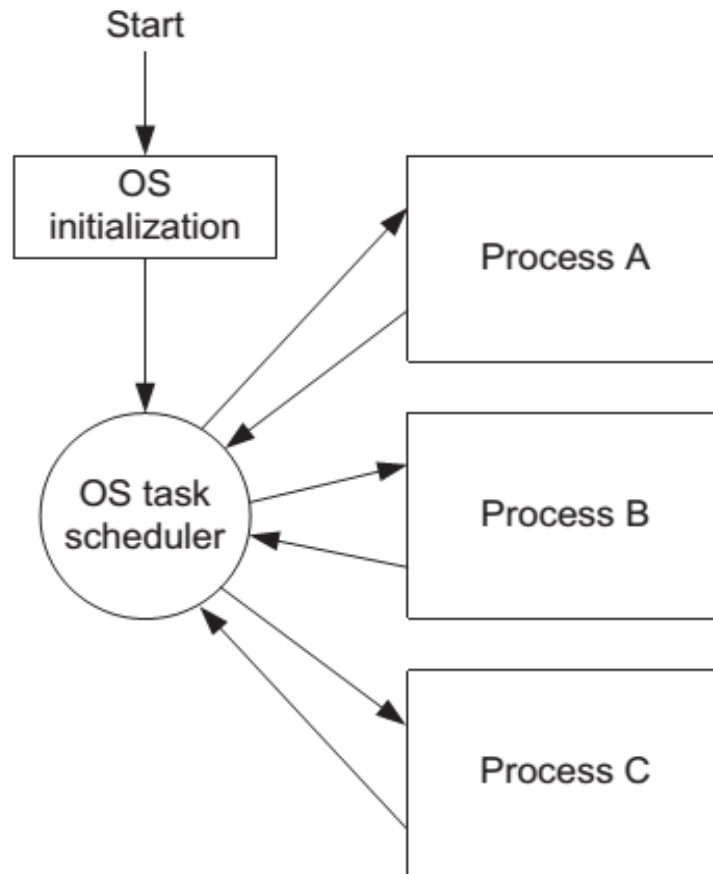
Interrupt Driven



❖ Interrupt Driven

- In applications that require lower power, processing can be carried out in interrupt service routines so that the processor can enter sleep mode when no processing is required.

Using RTOS



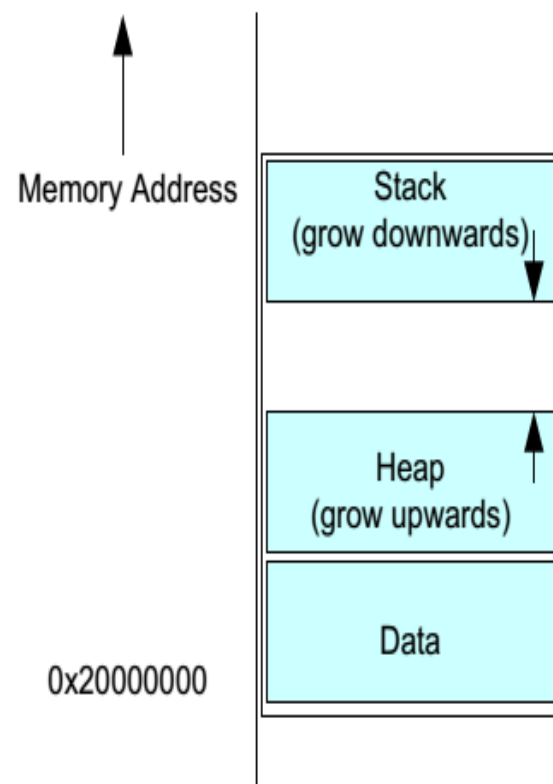
- ❖ For more complex applications, a real-time operating system (RTOS) can be used. An RTOS allows multiple processes to be executed by time slots and allocating one to each process.
- ❖ To use an RTOS, a timer is needed to generate regular interrupt requests. When each time slot ends, the timer generates an interrupt that triggers the RTOS task scheduler.

Data Types of C Programming

C and C99 (stdint.h) Data Type	Number of Bits	Range (Signed)	Range (Unsigned)
char, int8_t, uint8_t	8	−128 to 127	0 to 255
short int16_t, uint16_t	16	−32768 to 32767	0 to 65535
int, int32_t, uint32_t	32	−2147483648 to 2147483647	0 to 4294967295
long	32	−2147483648 to 2147483647	0 to 4294967295
long long, int64_t, uint64_t	64	− (2 ⁶³) to (2 ⁶³ − 1)	0 to (2 ⁶⁴ − 1)
float	32	−3.4028234 × 10 ³⁸ to 3.4028234 × 10 ³⁸	
double	64	−1.7976931348623157 × 10 ³⁰⁸ to 1.7976931348623157 × 10 ³⁰⁸	
long double	64	−1.7976931348623157 × 10 ³⁰⁸ to 1.7976931348623157 × 10 ³⁰⁸	
pointers	32	0x0 to 0xFFFFFFFF	
enum	8/16/32	Smallest possible data type, except when overridden by compiler option	
bool (C++ only), _Bool (C only)	8	True or false	
wchar_t	16	0 to 65535	

資料記憶體分類

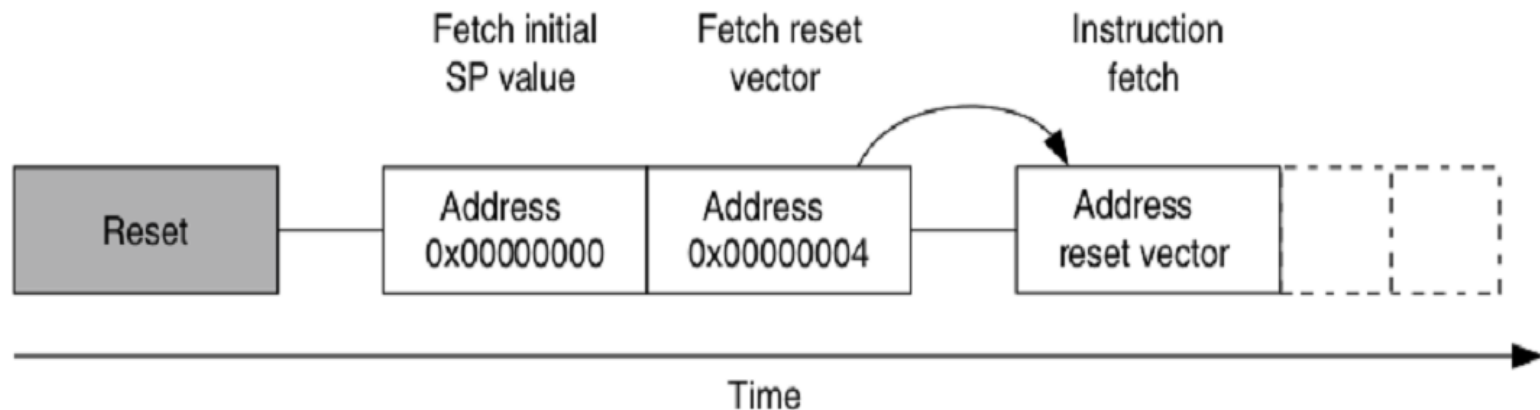
- ❖ 從程式的運作來看,資料記憶體可分為一般資料(data)區,堆疊(stack)區和堆積(heap)區. 為了確保程式可以正常使用這些不同記憶體, 通常在startup code中設定相關位址與大小.
 - 資料(data): 起始於記憶體的開始位址,一般用於儲存全域變數和靜態變數
 - 堆疊(stack): 因為堆疊記憶體的操作是基於滿遞減(full descending)的堆疊分配,所以通常會將堆疊指標設定在記憶體的最後位址.堆疊記憶體通常用於儲存函式的呼叫,例外處理時的暫存器備份,和區域變數等等.
 - 堆積(heap): 而堆積記憶體一般會設定在資料區塊的後面.通常是給C函式庫用來執行的動態分配記憶體空間.



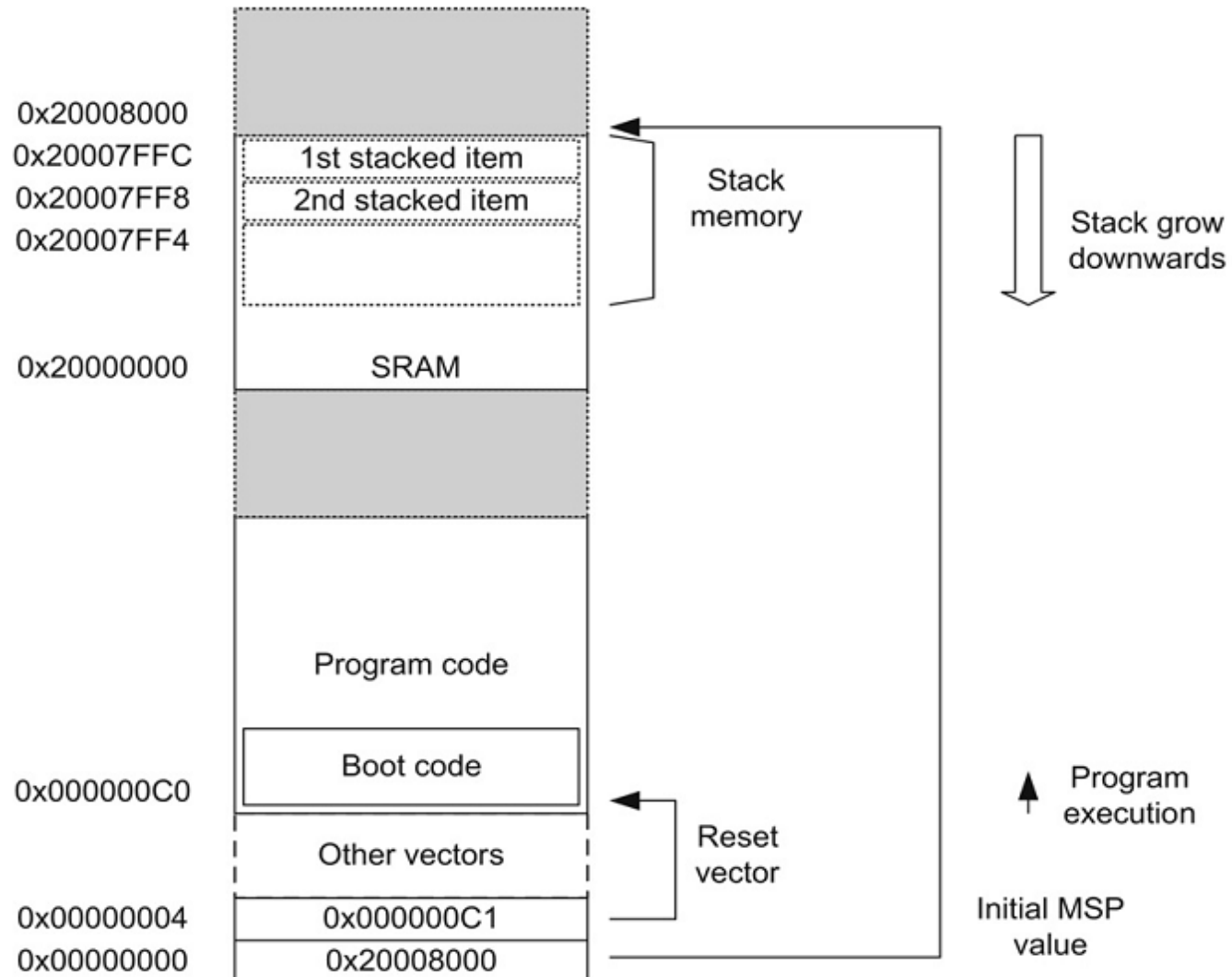
啟動流程

CPU Reset後程式的運作流程

- ❖ Reset會導致CPU重新初始化程式計數器(PC)的數值，程式計數器（PC）將會保留第一個欲執行程式的開始位址，該位址隨每個MCU core不同而異。
- ❖ 在離開Reset狀態後，M0所做的第一件事，就是讀取下列兩個位址的值：
 - 從位址 0×00000000 處取出MSP (Main Stack Point)的初始值放到SP暫存器(Stack Pointer)
 - 從位址 0×00000004 處取出reset 例外處理程式的起始點位址,放到程式計數暫存器(PC)，CPU隨即自這個位址開始執行程式。



M0 Program Structure



Reset handler routine

```
; Reset handler routine
Reset_Handler    PROC
                   EXPORT Reset_Handler            [WEAK]
                   IMPORT __main
                   IMPORT SystemInit
                   LDR     R0, =SystemInit
                   BLX     R0
                   LDR     R0, =__main
                   BX      R0
                   ENDP
```

Vector Table (向量表)

Exception number	IRQ number	Vector	Offset
47	31	IRQ31	0xBC
.		.	.
.		.	.
.		.	.
18	2	IRQ2	0x48
17	1	IRQ1	0x44
16	0	IRQ0	0x40
15	-1	SysTick	0x3C
14	-2	PendSV	0x38
13		Reserved	
12			
11	-5	SVCall	0x2C
10			
9			
8			
7		Reserved	
6			
5			
4			
3	-13	HardFault	0x10
2	-14	NMI	0x0C
1		Reset	0x08
		Initial SP value	0x04
			0x00

定義中斷執行函式進入位址

```
__Vectors      DCD      __initial_sp          ; Top of Stack
               DCD      Reset_Handler        ; Reset Handler
               DCD      NMI_Handler          ; NMI Handler
               DCD      HardFault_Handler    ; Hard Fault Handler
               DCD      0                     ; Reserved
               DCD      0                     ; Reserved
               DCD      0                     ; Reserved
               DCD      0                     ; Reserved
               DCD      0                     ; Reserved
               DCD      0                     ; Reserved
               DCD      0                     ; Reserved
               DCD      SVC_Handler          ; SVCcall Handler
               DCD      0                     ; Reserved
               DCD      0                     ; Reserved
               DCD      PendSV_Handler       ; PendSV Handler
               DCD      SysTick_Handler      ; SysTick Handler

               ; External Interrupts
               DCD      WWDG_IRQHandler      ; Window Watchdog
               DCD      PVD_VDDIO2_IRQHandler ; PVD through EXTI Line detect
               DCD      RTC_IRQHandler        ; RTC through EXTI Line
               DCD      FLASH_IRQHandler     ; FLASH
               DCD      RCC_CRIS_IRQHandler  ; RCC and CRS
               DCD      EXTI0_1_IRQHandler   ; EXTI Line 0 and 1
               DCD      EXTI2_3_IRQHandler   ; EXTI Line 2 and 3
               DCD      EXTI4_15_IRQHandler  ; EXTI Line 4 to 15
               DCD      TSC_IRQHandler       ; TS
               DCD      DMA1_Channel1_IRQHandler ; DMA1 Channel 1
               DCD      DMA1_Channel2_3_IRQHandler ; DMA1 Channel 2 and Channel 3
```

Dummy Exception Handlers (infinite loops which can be modified)

```
NMI_Handler      PROC
                  EXPORT NMI_Handler      [WEAK]
                  B       .
                  ENDP

HardFault_Handler\
                  PROC
                  EXPORT HardFault_Handler [WEAK]
                  B       .
                  ENDP

SVC_Handler      PROC
                  EXPORT SVC_Handler      [WEAK]
                  B       .
                  ENDP

PendSV_Handler   PROC
                  EXPORT PendSV_Handler   [WEAK]
                  B       .
                  ENDP

SysTick_Handler  PROC
                  EXPORT SysTick_Handler  [WEAK]
                  B       .
                  ENDP
```

組合語言語法

- ❖ ARM的組合語言(適用ARM組譯器; armasm)使用下列指令格式
 - 標記(label)被當作位址的參考值,在指令前使用標記,就可以得到指令的位址.
 - 第一個運算元通常被當作運算的目的地. 指令的類型不同,運算元的個數可能也不同,有些指令不需要任何運算元,有些只需要一個.
 - 分號後的文字為註解.

標記(label)

指令碼 運算元1, 運算元2, ; 註解
ADD R0, R1

- ❖ 如果使用GNU tool,其語法為
標記(label) :

指令碼 運算元1, 運算元2, /* 註解 */

- ❖ 立即值通常用”#”做前導

MOV R0, #0x12 ; set R0 = 0x12

組合語言語法

- ❖ PROC / FUNCTION指令讓組合語言的程式碼可以設計成函式的型式,方便程式呼叫. Use PROC to mark the start of functions. Each PROC directive must have a matching ENDP directive.
- ❖ Syntax
 - label PROC /FUNCTION
 - :
 - ENDP /ENDFUNC
- ❖ Example:
 - **[weak]:** 告訴連結器可以使用相同函式名稱的函式去取代被宣告為weak的函式.
 - funtest PROC
EXPORT funtest [weak]
B . ; branch to self (無限迴圈)
ENDP

CMSIS

CMSIS

- ❖ ARM為了讓程式設計師容易使用Cortex-M series MCU,所以定義了微控制器軟體介面標準(CMSIS , Cortex Microcontroller Software Interface Standard). 許多IC設計公司的驅動程式已使用CMSIS 標準,當成是函式庫的一部分,提供給客戶.
- ❖ CMSIS提供的功能包括:
 - 函式庫用來控制M0的一些共用功能,例如中斷和例外控制, System timer.
 - M0中一些特殊指令的標準化函式庫
 - 標準化 MCU的周邊存取使用方式

CMSIS標準化函式庫：中斷控制

- ❖ The CMSIS provides a number of functions for NVIC control including :

CMSIS interrupt control function	Description
void NVIC_EnableIRQ(IRQn_t IRQn)	Enable IRQn
void NVIC_DisableIRQ(IRQn_t IRQn)	Disable IRQn
uint32_t NVIC_GetPendingIRQ (IRQn_t IRQn)	Return true (1) if IRQn is pending
void NVIC_SetPendingIRQ (IRQn_t IRQn)	Set IRQn pending
void NVIC_ClearPendingIRQ (IRQn_t IRQn)	Clear IRQn pending status
void NVIC_SetPriority (IRQn_t IRQn, uint32_t priority)	Set priority for IRQn
uint32_t NVIC_GetPriority (IRQn_t IRQn)	Read priority of IRQn
void NVIC_SystemReset (void)	Reset the system

特殊指令的標準化函式庫

- ❖ The C code cannot directly access some Cortex-M0 instructions so the CMSIS provides the intrinsic functions to generate instructions .

Instruction	CMSIS intrinsic function
CPSIE I	void __enable_irq(void)
CPSID I	void __disable_irq(void)
ISB	void __ISB(void)
DSB	void __DSB(void)
DMB	void __DMB(void)
NOP	void __NOP(void)
REV	uint32_t __REV(uint32_t int value)
REV16	uint32_t __REV16(uint32_t int value)
REVSH	uint32_t __REVSH(uint32_t int value)
SEV	void __SEV(void)
WFE	void __WFE(void)
WFI	void __WFI(void)

CMSIS 標準化函式庫 access the special registers

Special register	Access	CMSIS function
PRIMASK	Read	uint32_t __get_PRIMASK (void)
	Write	void __set_PRIMASK (uint32_t value)
CONTROL	Read	uint32_t __get_CONTROL (void)
	Write	void __set_CONTROL (uint32_t value)
MSP	Read	uint32_t __get_MSP (void)
	Write	void __set_MSP (uint32_t TopOfMainStack)
PSP	Read	uint32_t __get_PSP (void)
	Write	void __set_PSP (uint32_t TopOfProcStack)

M0 Memory Map(記憶體配置)

- ❖ M0是32位元的MCU,所以具有4G bytes的記憶體定址空間,記憶體空間被定義成不同的區域,每個區域都有建議的用途. The Code, SRAM, and external RAM regions can hold programs. Execute Never (XN) means the processor prevents executing instruction .
- ❖ M0有一些標準的基本功能和除錯元件的控制暫存器,被設計在位址0xE0000000 – 0xE0100000 的固定記憶體位置,稱為私有周邊匯流排(Private peripheral bus, PPB).

Core Memory Mapped Register Space, i.e. NVIC (XN)	Device 511MB	0xE010 0000
External Peripherals (XN)	Private Peripheral Bus	0xE000 0000
External Memory	External Device 1GB	0xA000 0000
External Peripherals (XN)	External RAM 1GB	0x6000 0000
Data Memory (code can also be placed here)	Peripheral 500MB	0x4000 0000
Executable region for program code and data (vector table is fixed at address 0x0000 0000)	SRAM 500MB	0x2000 0000
	Code 500MB	0x0000 0000

XN – execute never

STM32F072 peripheral register boundary addresses

Bus	Boundary address	Size	Peripheral
	0x4800 1800 - 0x5FFF FFFF	~384 MB	Reserved
AHB2	0x4800 1400 - 0x4800 17FF	1 KB	GPIOF
	0x4800 1000 - 0x4800 13FF	1 KB	GPIOE
	0x4800 0C00 - 0x4800 0FFF	1 KB	GPIOD
	0x4800 0800 - 0x4800 0BFF	1 KB	GPIOC
	0x4800 0400 - 0x4800 07FF	1 KB	GPIOB
	0x4800 0000 - 0x4800 03FF	1 KB	GPIOA
	0x4002 4400 - 0x47FF FFFF	~128 MB	Reserved
AHB1	0x4002 4000 - 0x4002 43FF	1 KB	TSC
	0x4002 3400 - 0x4002 3FFF	3 KB	Reserved
	0x4002 3000 - 0x4002 33FF	1 KB	CRC
	0x4002 2400 - 0x4002 2FFF	3 KB	Reserved
	0x4002 2000 - 0x4002 23FF	1 KB	Flash memory interface
	0x4002 1400 - 0x4002 1FFF	3 KB	Reserved
	0x4002 1000 - 0x4002 13FF	1 KB	RCC
	0x4002 0400 - 0x4002 0FFF	3 KB	Reserved
	0x4002 0000 - 0x4002 03FF	1 KB	DMA

APB	0x4001 5C00 - 0x4001 7FFF	9 KB	Reserved
	0x4001 5800 - 0x4001 5BFF	1 KB	DBGMCU
	0x4001 4C00 - 0x4001 57FF	3 KB	Reserved
	0x4001 4800 - 0x4001 4BFF	1 KB	TIM17
	0x4001 4400 - 0x4001 47FF	1 KB	TIM16
	0x4001 4000 - 0x4001 43FF	1 KB	TIM15
	0x4001 3C00 - 0x4001 3FFF	1 KB	Reserved
	0x4001 3800 - 0x4001 3BFF	1 KB	USART1
	0x4001 3400 - 0x4001 37FF	1 KB	Reserved
	0x4001 3000 - 0x4001 33FF	1 KB	SPI1/I2S1
	0x4001 2C00 - 0x4001 2FFF	1 KB	TIM1
	0x4001 2800 - 0x4001 2BFF	1 KB	Reserved
	0x4001 2400 - 0x4001 27FF	1 KB	ADC
	0x4001 0800 - 0x4001 23FF	7 KB	Reserved
	0x4001 0400 - 0x4001 07FF	1 KB	EXTI
	0x4001 0000 - 0x4001 03FF	1 KB	SYSCFG + COMP

APB	0x4000 5800 - 0x4000 5BFF	1 KB	I2C2
	0x4000 5400 - 0x4000 57FF	1 KB	I2C1
	0x4000 5000 - 0x4000 53FF	1 KB	Reserved
	0x4000 4C00 - 0x4000 4FFF	1 KB	USART4
	0x4000 4800 - 0x4000 4BFF	1 KB	USART3
	0x4000 4400 - 0x4000 47FF	1 KB	USART2
	0x4000 3C00 - 0x4000 43FF	2 KB	Reserved
	0x4000 3800 - 0x4000 3BFF	1 KB	SPI2
	0x4000 3400 - 0x4000 37FF	1 KB	Reserved
	0x4000 3000 - 0x4000 33FF	1 KB	IWDG
	0x4000 2C00 - 0x4000 2FFF	1 KB	WWDG
	0x4000 2800 - 0x4000 2BFF	1 KB	RTC
	0x4000 2400 - 0x4000 27FF	1 KB	Reserved
	0x4000 2000 - 0x4000 23FF	1 KB	TIM14
	0x4000 1800 - 0x4000 1FFF	2 KB	Reserved
	0x4000 1400 - 0x4000 17FF	1 KB	TIM7
	0x4000 1000 - 0x4000 13FF	1 KB	TIM6
	0x4000 0800 - 0x4000 0FFF	2 KB	Reserved
	0x4000 0400 - 0x4000 07FF	1 KB	TIM3
	0x4000 0000 - 0x4000 03FF	1 KB	TIM2

Define the Peripheral Register Set as a data structure

- ❖ To simplify the code, we can define the peripheral register set as a data structure and define the specific peripheral name as a memory pointer to this data structure.
- ❖ For example: General Purpose I/O control register

```
typedef struct
{
    __IO uint32_t MODER;           /*!< GPIO port mode register,           Address offset: 0x00 */
    __IO uint32_t OTYPER;          /*!< GPIO port output type register,      Address offset: 0x04 */
    __IO uint32_t OSPEEDR;         /*!< GPIO port output speed register,     Address offset: 0x08 */
    __IO uint32_t PUPDR;           /*!< GPIO port pull-up/pull-down register, Address offset: 0x0C */
    __IO uint32_t IDR;             /*!< GPIO port input data register,       Address offset: 0x10 */
    __IO uint32_t ODR;             /*!< GPIO port output data register,      Address offset: 0x14 */
    __IO uint32_t BSRR;            /*!< GPIO port bit set/reset register,    Address offset: 0x1A */
    __IO uint32_t LCKR;            /*!< GPIO port configuration lock register, Address offset: 0x1C */
    __IO uint32_t AFR[2];          /*!< GPIO alternate function low register, Address offset: 0x20-0x24 */
    __IO uint32_t BRR;             /*!< GPIO bit reset register,            Address offset: 0x28 */
} GPIO_TypeDef;
```

Ex: 設定GPIOA暫存器的值

Ex: 設定GPIOA暫存器的值

```
//GPIOA_BASE = 0x4800 0000 ;
```

```
#define GPIOA    ((GPIO_TypeDef *) GPIOA_BASE )
```

```
GPIOA->MODER &= ~ (0x3ul << 2*5);      // set GPIOA pin 5 as output pin
```

```
GPIOA->MODER |= (0x1ul << 2*5);
```

C語言中的ul

- ❖ C語言中的數值常數有整數常數、浮點常數，只有數值常數才有後綴說明，後綴說明不區分字母大小寫。
- ❖ 整數常數的表示形式有十進制形式、以0x開頭的十六進制形式。
 - 對整數常數進行類型轉換的後綴只有：u或U（unsigned）、l或L（long）、u/U與l/L的組合（如：ul、lu、Lu等）。
 - 例：100ul；0x12ul。
- ❖ $0x1ul \ll 16$
 - 把沒有符號的整數1 向左移位16次變成0x00010000.

volatile關鍵字

- ❖ 宣告變數為volatile,是用來告訴編譯器 (Compiler) 不要對該變數做任何最佳化操作，凡牽涉讀取該volatile變數的操作，保證會到該變數的實體位址讀取內容。
- ❖ Volatile關鍵字的應用範疇
 - 1. 硬體周邊所有暫存器。
 - 2. 共用的全域變數。
 - 3. 中斷服務函式所使用的全域變數。
- ❖ `#define __IO volatile` `/*!< Defines 'read / write' permissions */`

Peripheral pointer declaration (I)

```
#define TIM2          ((TIM_TypeDef *) TIM2_BASE)
#define TIM3          ((TIM_TypeDef *) TIM3_BASE)
#define TIM6          ((TIM_TypeDef *) TIM6_BASE)
#define TIM7          ((TIM_TypeDef *) TIM7_BASE)
#define TIM14         ((TIM_TypeDef *) TIM14_BASE)
#define RTC           ((RTC_TypeDef *) RTC_BASE)
#define WWDG          ((WWDG_TypeDef *) WWDG_BASE)
#define IWDG          ((IWDG_TypeDef *) IWDG_BASE)
#define USART2        ((USART_TypeDef *) USART2_BASE)
#define USART3        ((USART_TypeDef *) USART3_BASE)
#define USART4        ((USART_TypeDef *) USART4_BASE)
#define I2C1          ((I2C_TypeDef *) I2C1_BASE)
#define I2C2          ((I2C_TypeDef *) I2C2_BASE)
#define CAN           ((CAN_TypeDef *) CAN_BASE)
#define CRS           ((CRS_TypeDef *) CRS_BASE)
#define PWR           ((PWR_TypeDef *) PWR_BASE)
#define DAC1          ((DAC_TypeDef *) DAC_BASE)
#define DAC           ((DAC_TypeDef *) DAC_BASE) /* Kept for legacy purpose */
#define CEC           ((CEC_TypeDef *) CEC_BASE)
#define SYSCFG        ((SYSCFG_TypeDef *) SYSCFG_BASE)
#define COMP1         ((COMP_TypeDef *) COMP_BASE)
#define COMP2         ((COMP_TypeDef *) (COMP_BASE + 0x00000002))
#define COMP12_COMMON ((COMP_Common_TypeDef *) COMP_BASE)
#define COMP          ((COMP1_2_TypeDef *) COMP_BASE) /* Kept for legacy purpose */
#define EXTI          ((EXTI_TypeDef *) EXTI_BASE)
#define ADC1          ((ADC_TypeDef *) ADC1_BASE)
#define ADC1_COMMON   ((ADC_Common_TypeDef *) ADC_BASE)
#define ADC           ((ADC_Common_TypeDef *) ADC_BASE) /* Kept for legacy purpose */
```

Peripheral pointer declaration (II)

```
#define TIM1 ((TIM_TypeDef *) TIM1_BASE)
#define SPI1 ((SPI_TypeDef *) SPI1_BASE)
#define SPI2 ((SPI_TypeDef *) SPI2_BASE)
#define USART1 ((USART_TypeDef *) USART1_BASE)
#define TIM15 ((TIM_TypeDef *) TIM15_BASE)
#define TIM16 ((TIM_TypeDef *) TIM16_BASE)
#define TIM17 ((TIM_TypeDef *) TIM17_BASE)
#define DBGMCU ((DBGMCU_TypeDef *) DBGMCU_BASE)
#define DMA1 ((DMA_TypeDef *) DMA1_BASE)
#define DMA1_Channel1 ((DMA_Channel_TypeDef *) DMA1_Channel1_BASE)
#define DMA1_Channel2 ((DMA_Channel_TypeDef *) DMA1_Channel2_BASE)
#define DMA1_Channel3 ((DMA_Channel_TypeDef *) DMA1_Channel3_BASE)
#define DMA1_Channel4 ((DMA_Channel_TypeDef *) DMA1_Channel4_BASE)
#define DMA1_Channel5 ((DMA_Channel_TypeDef *) DMA1_Channel5_BASE)
#define DMA1_Channel6 ((DMA_Channel_TypeDef *) DMA1_Channel6_BASE)
#define DMA1_Channel7 ((DMA_Channel_TypeDef *) DMA1_Channel7_BASE)
#define FLASH ((FLASH_TypeDef *) FLASH_R_BASE)
#define OB ((OB_TypeDef *) OB_BASE)
#define RCC ((RCC_TypeDef *) RCC_BASE)
#define CRC ((CRC_TypeDef *) CRC_BASE)
#define TSC ((TSC_TypeDef *) TSC_BASE)
#define GPIOA ((GPIO_TypeDef *) GPIOA_BASE)
#define GPIOB ((GPIO_TypeDef *) GPIOB_BASE)
#define GPIOC ((GPIO_TypeDef *) GPIOC_BASE)
#define GPIOD ((GPIO_TypeDef *) GPIOD_BASE)
#define GPIOE ((GPIO_TypeDef *) GPIOE_BASE)
#define GPIOF ((GPIO_TypeDef *) GPIOF_BASE)
#define USB ((USB_TypeDef *) USB_BASE)
```

General-purpose I/O

General Purpose I/O registers

```
typedef struct
{
    __IO uint32_t MODER;          /*!< GPIO port mode register,           Address offset: 0x00 */
    __IO uint32_t OTYPER;         /*!< GPIO port output type register,      Address offset: 0x04 */
    __IO uint32_t OSPEEDR;        /*!< GPIO port output speed register,     Address offset: 0x08 */
    __IO uint32_t PUPDR;          /*!< GPIO port pull-up/pull-down register, Address offset: 0x0C */
    __IO uint32_t IDR;            /*!< GPIO port input data register,       Address offset: 0x10 */
    __IO uint32_t ODR;            /*!< GPIO port output data register,      Address offset: 0x14 */
    __IO uint32_t BSRR;           /*!< GPIO port bit set/reset register,    Address offset: 0x1A */
    __IO uint32_t LCKR;           /*!< GPIO port configuration lock register, Address offset: 0x1C */
    __IO uint32_t AFR[2];         /*!< GPIO alternate function low register, Address offset: 0x20-0x24 */
    __IO uint32_t BRR;            /*!< GPIO bit reset register,            Address offset: 0x28 */
} GPIO_TypeDef;
```

List of abbreviations for registers

- ❖ **read/write (rw)** : Software can read and write to these bits.
- ❖ **read-only (r)** : Software can only read these bits.
- ❖ **write-only (w)** : Software can only write to this bit. Reading the bit returns the reset value.
- ❖ **read/clear (rc_w1)** : Software can read as well as clear this bit by writing 1. Writing '0' has no effect on the bit value.
- ❖ **read/clear (rc_w0)** : Software can read as well as clear this bit by writing 0. Writing '1' has no effect on the bit value.
- ❖ **read/clear by read (rc_r)** : Software can read this bit. Reading this bit automatically clears it to '0'. Writing '0' has no effect on the bit value.
- ❖ **read/set (rs)** : Software can read as well as set this bit. Writing '0' has no effect on the bit value.
- ❖ **Reserved (Res.)** : Reserved bit, must be kept at reset value.

AHB peripheral clock enable register : RCC_AHBENR

- ❖ Address offset: 0x14
- ❖ Reset value: 0x0000 0014
- ❖ Note: When the peripheral clock is not active, the peripheral register values may not be readable by software and the returned value is always 0x0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	TSCEN	Res.	IOPF EN	IOPE EN	IOPD EN	IOPC EN	IOPB EN	IOPA EN	Res.
							rw		rw	rw	rw	rw	rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC EN	Res.	FLITF EN	Res.	SRAM EN	DMA2 EN	DMA EN
									rw		rw		rw	rw	rw

Bit 17 IOPAEN: I/O port A clock enable
Set and cleared by software.
0: I/O port A clock disabled
1: I/O port A clock enabled

Bit 18 IOPBEN: I/O port B clock enable
Set and cleared by software.
0: I/O port B clock disabled
1: I/O port B clock enabled

Bit 22 IOPFEN: I/O port F clock enable
Set and cleared by software.
0: I/O port F clock disabled
1: I/O port F clock enabled

Bit 21 IOPEEN: I/O port E clock enable
Set and cleared by software.
0: I/O port E clock disabled
1: I/O port E clock enabled

Bit 20 IOPDEN: I/O port D clock enable
Set and cleared by software.
0: I/O port D clock disabled
1: I/O port D clock enabled

Bit 19 IOPCEN: I/O port C clock enable
Set and cleared by software.
0: I/O port C clock disabled
1: I/O port C clock enabled

GPIOx_MODER: mode register (x = A..F)

- ❖ Address offset: 0x00
- ❖ Reset values:
 - 0x2800 0000 for port A, PA.14 (SWCLK) ,PA.13(SWDIO) = Alternate function mode for SWD debug port
 - 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y+1:2y **MODERy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O mode.

- 00: Input mode (reset state)
- 01: General purpose output mode
- 10: Alternate function mode
- 11: Analog mode

GPIOx_OSPEEDR: output speed register (x = A..F)

- ❖ Address offset: 0x08
- ❖ Reset value:
 - 0x0C00 0000 for port A , PA.13(SWDIO) = High speed
 - 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OSPEEDR15 [1:0]		OSPEEDR14 [1:0]		OSPEEDR13 [1:0]		OSPEEDR12 [1:0]		OSPEEDR11 [1:0]		OSPEEDR10 [1:0]		OSPEEDR9 [1:0]		OSPEEDR8 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OSPEEDR7 [1:0]		OSPEEDR6 [1:0]		OSPEEDR5 [1:0]		OSPEEDR4 [1:0]		OSPEEDR3 [1:0]		OSPEEDR2 [1:0]		OSPEEDR1 [1:0]		OSPEEDR0 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y+1:2y **OSPEEDRy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O output speed.

x0: Low speed

01: Medium speed

11: High speed

Note: Refer to the device datasheet for the frequency specifications and the power supply and load conditions for each speed.

GPIOx_IDR: input data register (x = A..F)

- ❖ The input data register captures the data present on the I/O pin at every AHB clock cycle.
- ❖ Address offset: 0x10
- ❖ Reset value: 0x0000 XXXX (where X means undefined)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **IDRy**: Port input data bit (y = 0..15)

These bits are read-only. They contain the input value of the corresponding I/O port.

GPIOx_ODR : output data register (x = A..F)

- ❖ The value written to the output data register is output on the I/O pin.
- ❖ Address offset: 0x14
- ❖ Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODRy**: Port output data bit (y = 0..15)

These bits can be read and written by software.

Note: For atomic bit set/reset, the ODR bits can be individually set and/or reset by writing to the GPIOx_BSRR or GPIOx_BRR registers (x = A..F).

GPIOx_AFRL: alternate function low register (x = A..F)

- ❖ Address offset: 0x20
- ❖ Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFR7[3:0]				AFR6[3:0]				AFR5[3:0]				AFR4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFR3[3:0]				AFR2[3:0]				AFR1[3:0]				AFR0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFRy[3:0]**: Alternate function selection for port x pin y (y = 0..7)

These bits are written by software to configure alternate function I/Os

AFRy selection:

0000: AF0

0001: AF1

0010: AF2

0011: AF3

0100: AF4

0101: AF5

0110: AF6

0111: AF7

1000: Reserved

1001: Reserved

1010: Reserved

1011: Reserved

1100: Reserved

1101: Reserved

1110: Reserved

1111: Reserved

GPIOx_AFRH: alternate function high register (x = A..F)

- ❖ Address offset: 0x24
- ❖ Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFR15[3:0]				AFR14[3:0]				AFR13[3:0]				AFR12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFR11[3:0]				AFR10[3:0]				AFR9[3:0]				AFR8[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFRy[3:0]**: Alternate function selection for port x pin y (y = 8..15)

These bits are written by software to configure alternate function I/Os

AFRy selection:

0000: AF0

0001: AF1

0010: AF2

0011: AF3

0100: AF4

0101: AF5

0110: AF6

0111: AF7

1000: Reserved

1001: Reserved

1010: Reserved

1011: Reserved

1100: Reserved

1101: Reserved

1110: Reserved

1111: Reserved

Alternate functions GPIOA_AFR for port A

Pin name	AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7
PA0	-	USART2_CTS	TIM2_CH1_ETR	TSC_G1_IO1	USART4_TX	-	-	COMP1_OUT
PA1	EVENTOUT	USART2_RTS	TIM2_CH2	TSC_G1_IO2	USART4_RX	TIM15_CH1N	-	-
PA2	TIM15_CH1	USART2_TX	TIM2_CH3	TSC_G1_IO3	-	-	-	COMP2_OUT
PA3	TIM15_CH2	USART2_RX	TIM2_CH4	TSC_G1_IO4	-	-	-	-
PA4	SPI1_NSS, I2S1_WS	USART2_CK	-	TSC_G2_IO1	TIM14_CH1	-	-	-
PA5	SPI1_SCK, I2S1_CK	CEC	TIM2_CH1_ETR	TSC_G2_IO2	-	-	-	-
PA6	SPI1_MISO, I2S1_MCK	TIM3_CH1	TIM1_BKIN	TSC_G2_IO3	USART3_CTS	TIM16_CH1	EVENTOUT	COMP1_OUT
PA7	SPI1_MOSI, I2S1_SD	TIM3_CH2	TIM1_CH1N	TSC_G2_IO4	TIM14_CH1	TIM17_CH1	EVENTOUT	COMP2_OUT
PA8	MCO	USART1_CK	TIM1_CH1	EVENTOUT	CRS_SYNC	-	-	-
PA9	TIM15_BKIN	USART1_TX	TIM1_CH2	TSC_G4_IO1	-	-	-	-
PA10	TIM17_BKIN	USART1_RX	TIM1_CH3	TSC_G4_IO2	-	-	-	-
PA11	EVENTOUT	USART1_CTS	TIM1_CH4	TSC_G4_IO3	CAN_RX	-	-	COMP1_OUT
PA12	EVENTOUT	USART1_RTS	TIM1_ETR	TSC_G4_IO4	CAN_TX	-	-	COMP2_OUT
PA13	SWDIO	IR_OUT	USB_NOE	-	-	-	-	-
PA14	SWCLK	USART2_TX	-	-	-	-	-	-
PA15	SPI1_NSS, I2S1_WS	USART2_RX	TIM2_CH1_ETR	EVENTOUT	USART4_RTS	-	-	-

Alternate functions GPIOB_AFR for port B

Pin name	AF0	AF1	AF2	AF3	AF4	AF5
PB0	EVENTOUT	TIM3_CH3	TIM1_CH2N	TSC_G3_IO2	USART3_CK	-
PB1	TIM14_CH1	TIM3_CH4	TIM1_CH3N	TSC_G3_IO3	USART3_RTS	-
PB2	-	-	-	TSC_G3_IO4	-	-
PB3	SPI1_SCK, I2S1_CK	EVENTOUT	TIM2_CH2	TSC_G5_IO1	-	-
PB4	SPI1_MISO, I2S1_MCK	TIM3_CH1	EVENTOUT	TSC_G5_IO2	-	TIM17_BKIN
PB5	SPI1_MOSI, I2S1_SD	TIM3_CH2	TIM16_BKIN	I2C1_SMBA	-	-
PB6	USART1_TX	I2C1_SCL	TIM16_CH1N	TSC_G5_IO3	-	-
PB7	USART1_RX	I2C1_SDA	TIM17_CH1N	TSC_G5_IO4	USART4_CTS	-
PB8	CEC	I2C1_SCL	TIM16_CH1	TSC_SYNC	CAN_RX	-
PB9	IR_OUT	I2C1_SDA	TIM17_CH1	EVENTOUT	CAN_TX	SPI2_NSS, I2S2_WS
PB10	CEC	I2C2_SCL	TIM2_CH3	TSC_SYNC	USART3_TX	SPI2_SCK, I2S2_CK
PB11	EVENTOUT	I2C2_SDA	TIM2_CH4	TSC_G6_IO1	USART3_RX	-
PB12	SPI2_NSS, I2S2_WS	EVENTOUT	TIM1_BKIN	TSC_G6_IO2	USART3_CK	TIM15_BKIN
PB13	SPI2_SCK, I2S2_CK	-	TIM1_CH1N	TSC_G6_IO3	USART3_CTS	I2C2_SCL
PB14	SPI2_MISO, I2S2_MCK	TIM15_CH1	TIM1_CH2N	TSC_G6_IO4	USART3_RTS	I2C2_SDA
PB15	SPI2_MOSI, I2S2_SD	TIM15_CH2	TIM1_CH3N	TIM15_CH1N	-	-

Alternate functions GPIOC_AFR for port C

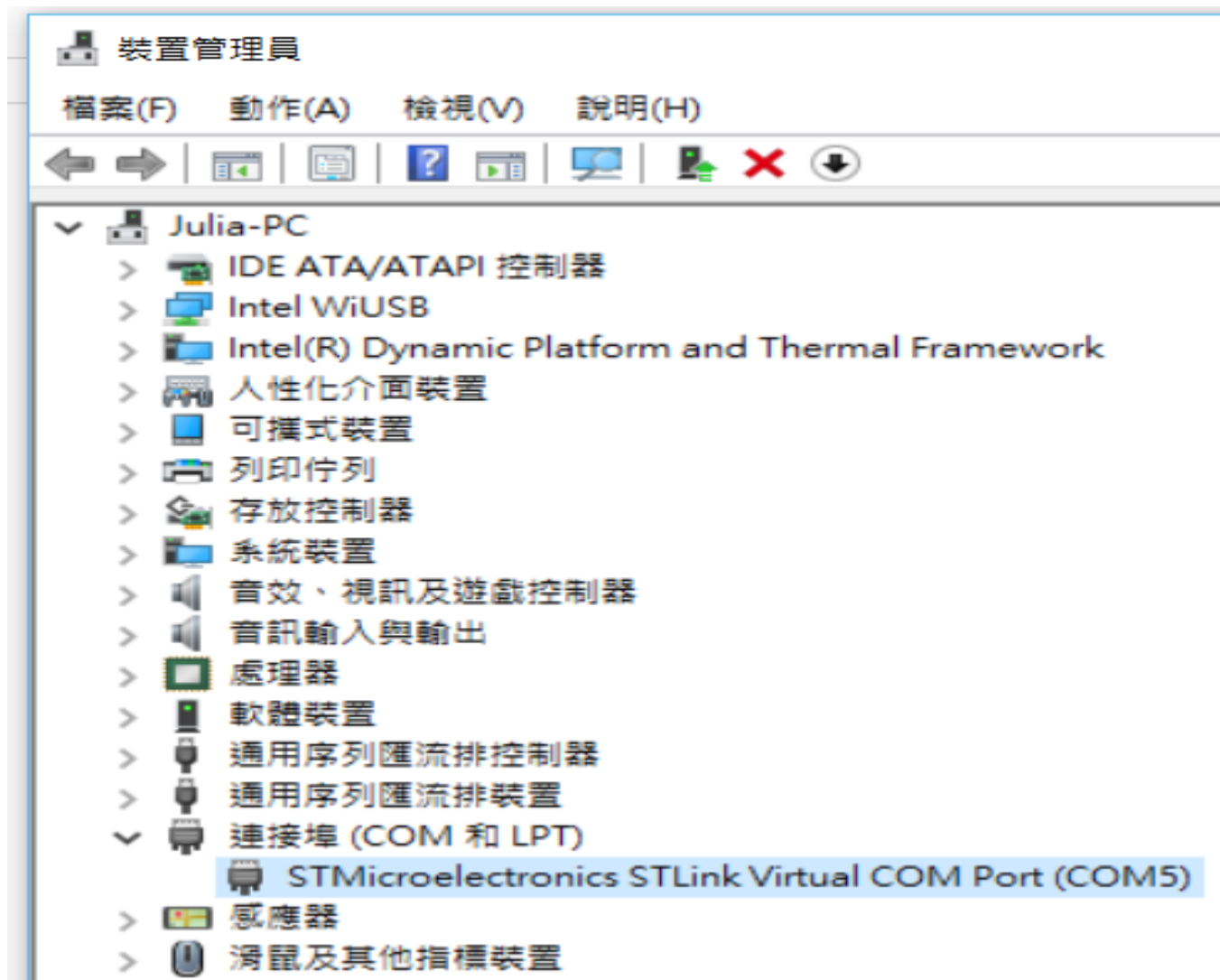
Pin name	AF0	AF1
PC0	EVENTOUT	-
PC1	EVENTOUT	-
PC2	EVENTOUT	SPI2_MISO, I2S2_MCK
PC3	EVENTOUT	SPI2_MOSI, I2S2_SD
PC4	EVENTOUT	USART3_TX
PC5	TSC_G3_IO1	USART3_RX
PC6	TIM3_CH1	-
PC7	TIM3_CH2	-
PC8	TIM3_CH3	-
PC9	TIM3_CH4	-
PC10	USART4_TX	USART3_TX
PC11	USART4_RX	USART3_RX
PC12	USART4_CK	USART3_CK
PC13	-	-
PC14	-	-
PC15	-	-

Alternate functions GPIOD_AFR for port D

Pin name	AF0	AF1
PD0	CAN_RX	SPI2_NSS, I2S2_WS
PD1	CAN_TX	SPI2_SCK, I2S2_CK
PD2	TIM3_ETR	USART3_RTS
PD3	USART2_CTS	SPI2_MISO, I2S2_MCK
PD4	USART2_RTS	SPI2_MOSI, I2S2_SD
PD5	USART2_TX	-
PD6	USART2_RX	-
PD7	USART2_CK	-
PD8	USART3_TX	-
PD9	USART3_RX	-
PD10	USART3_CK	-
PD11	USART3_CTS	-
PD12	USART3_RTS	TSC_G8_IO1
PD13	-	TSC_G8_IO2
PD14	-	TSC_G8_IO3
PD15	CRS_SYNC	TSC_G8_IO4

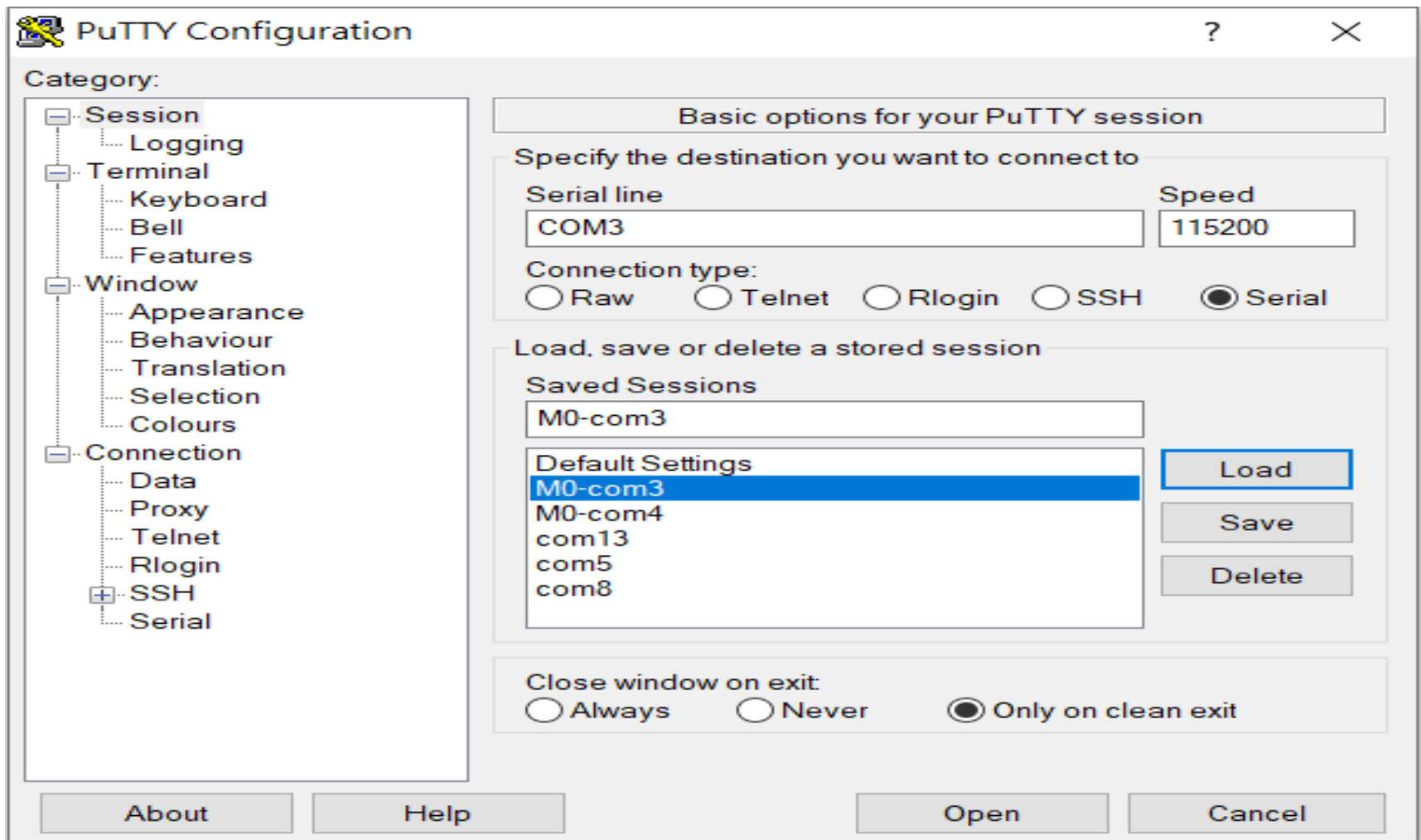
Setup Putty

ST-LINK/V2-1 installation

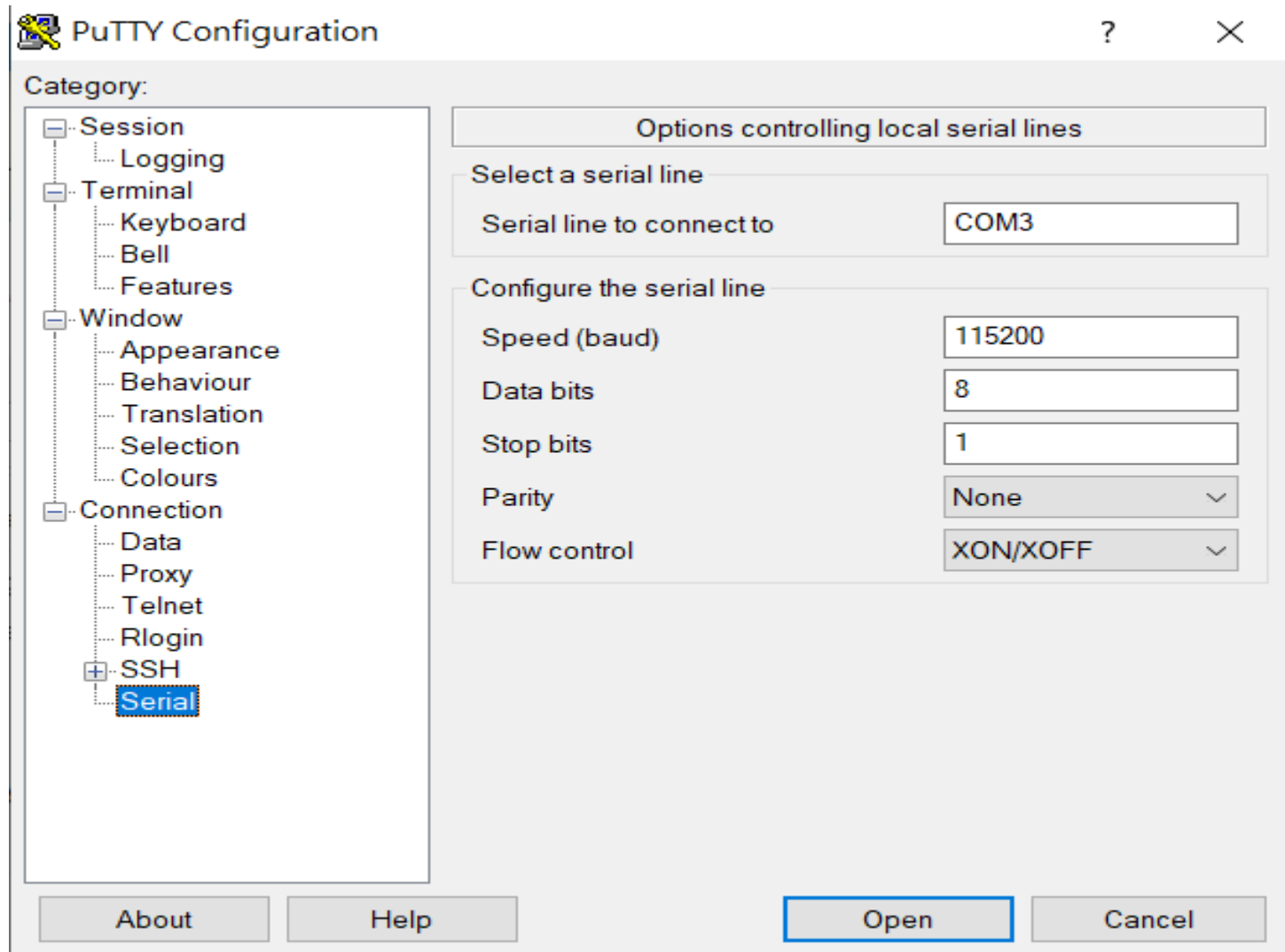


Install PUTTY (com port terminal)

❖ 執行 PUTTY.exe, Select serial port and change speed



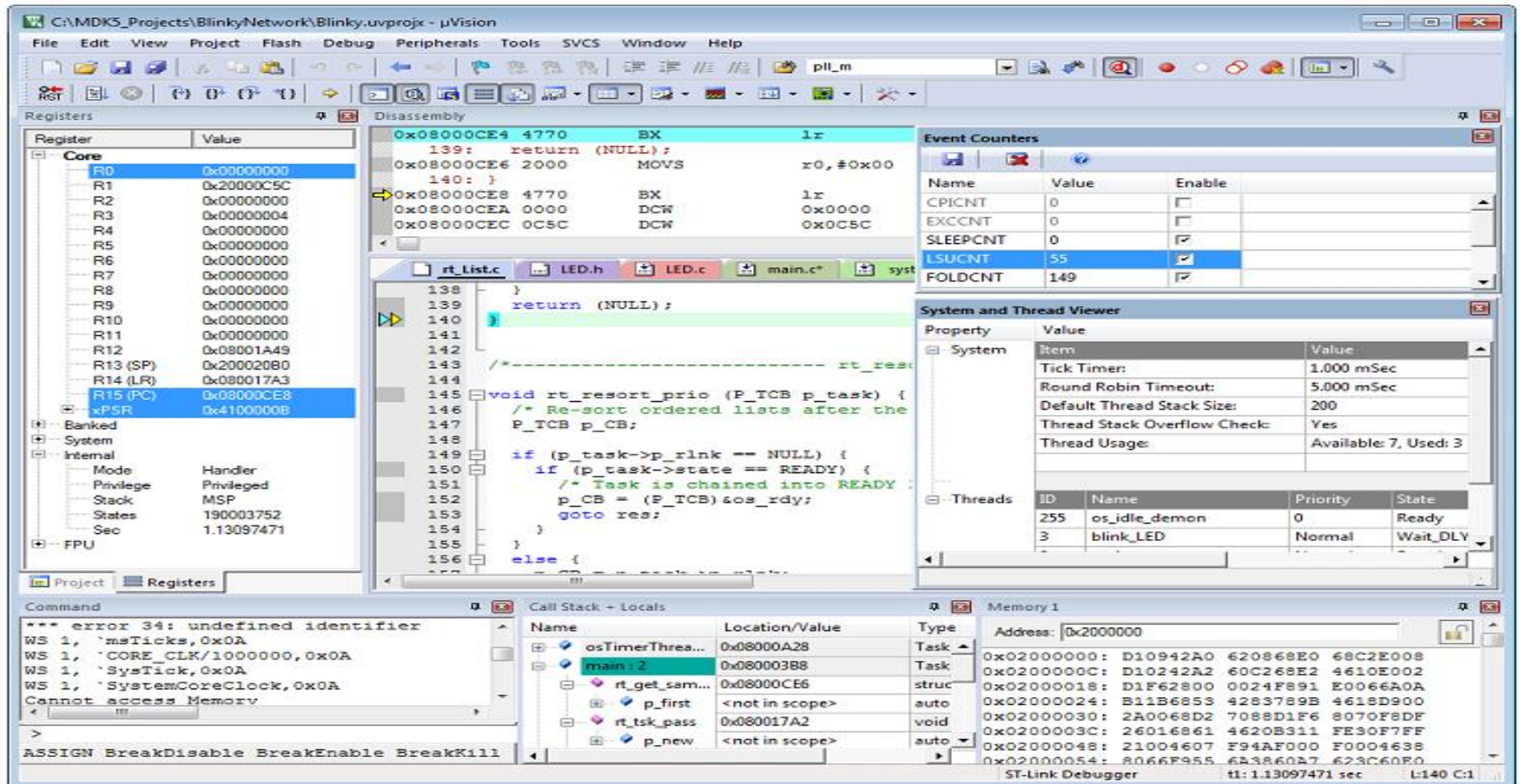
PuTTY Serial port configuration



Debug features

Start debugging

- ❖  Start debugging on hardware. From the toolbar, select Start/Stop Debug Session. During the start of a debugging session, μ Vision loads the application, executes the startup code, and stops at the main C function.
- ❖  Click Run on the toolbar.



The screenshot displays the μ Vision IDE interface for a project named "Blinky.uvprojx". The main window shows the C source code for "rt_list.c", with a breakpoint set at line 139. The left pane shows the Register window, displaying the Core registers (R0-R15) and the xPSR register. The right pane shows the Event Counters window, listing counters like CPICNT, EXCCNT, SLEEP CNT, LSUCNT (55), and FOLDCNT (149). The bottom pane shows the System and Thread Viewer window, displaying system properties such as Tick Timer (1.000 mSec) and Thread Usage (Available: 7, Used: 3). The bottom-most pane shows the Command window with error messages and the Call Stack window with the current stack frame.

Registers:

Register	Value
R0	0x00000000
R1	0x20000C5C
R2	0x00000000
R3	0x00000004
R4	0x00000000
R5	0x00000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x0801A49
R13 (SP)	0x20002080
R14 (LR)	0x08017A3
R15 (PC)	0x08000CE8
xPSR	0x4100000B

Event Counters:

Name	Value	Enable
CPICNT	0	☐
EXCCNT	0	☐
SLEEP CNT	0	☐
LSUCNT	55	☒
FOLDCNT	149	☒

System and Thread Viewer:

Property	Value
System	
Item	Value
Tick Timer:	1.000 mSec
Round Robin Timeout:	5.000 mSec
Default Thread Stack Size:	200
Thread Stack Overflow Check:	Yes
Thread Usage:	Available: 7, Used: 3

Threads:

ID	Name	Priority	State
255	os_idle_demon	0	Ready
3	blink_LED	Normal	Wait_DLY

Call Stack + Locals:

Name	Location/Value	Type
osTimerThrea...	0x08000A28	Task
main: 2	0x080003B8	Task
rt_get_sam...	0x08000CE6	struct
p_first	<not in scope>	auto
rt_tsk_pass	0x080017A2	void
p_new	<not in scope>	auto

Memory 1:

Address	Value
0x02000000	D10942A0 620868E0 68C2E008
0x0200000C	D10242A2 60C268E2 4610E002
0x02000018	D1F62800 0024F891 E0066A0A
0x02000024	B11B6853 4283789B 4618D900
0x02000030	2A0068D2 7088D1F6 8070F8DF
0x0200003C	26016861 4620B311 FE30F7FF
0x02000048	21004607 F94AF000 F0004638
0x02000054	8066F955 6A3860A7 623C60E0

Command:

```
*** error 34: undefined identifier
WS 1, 'msTicks,0x0A
WS 1, 'CORE_CLK/1000000,0x0A
WS 1, 'SysTick,0x0A
WS 1, 'SystemCoreClock,0x0A
Cannot access Memory
>
ASSIGN BreakDisable BreakEnable BreakKill
```

Debug Toolbar

❖ The debug toolbar provides quick access to many debugging commands such as :

 **Step** steps through the program and into function calls.

 **Step Over** steps through the program and over function calls.

 **Step Out** steps out of the current function.

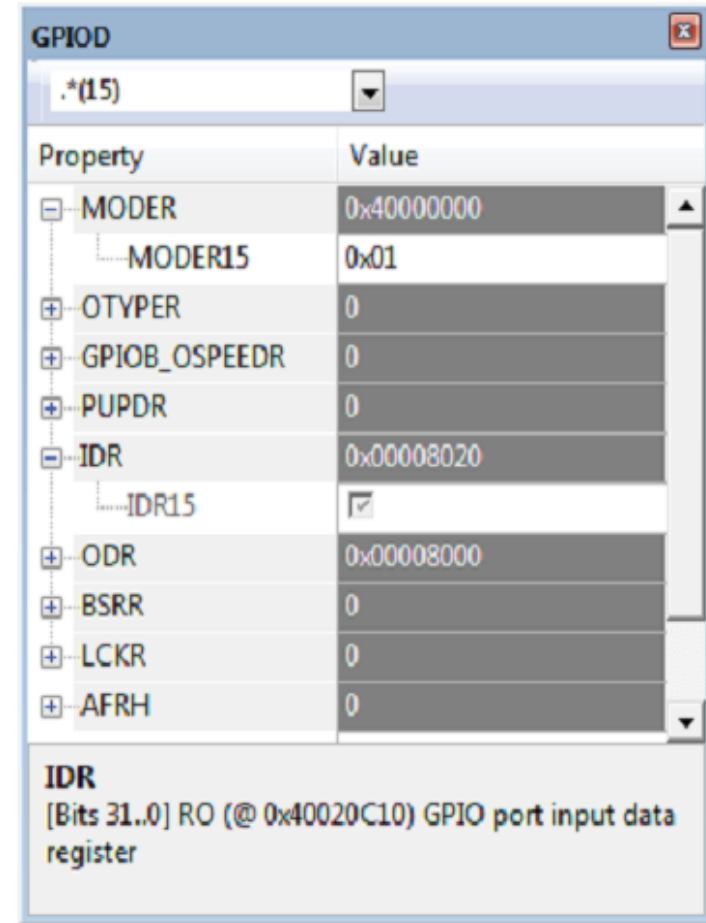
 **Stop** halts program execution.

 **Reset** performs a CPU reset.

 **Show** to the statement that executes next (current PC location).

Peripheral Registers

- ❖ Peripheral registers are memory mapped registers to which a processor can write to and read from to control a peripheral. The menu Peripherals provides access to Core Peripherals, such as the Nested Vector Interrupt Controller or the System Tick Timer. You can access device peripheral registers using the System Viewer.
- ❖ System Viewer windows display information about device peripheral registers and values are updated periodically. You can change property values while debugging.
- ❖  Open a peripheral register from the toolbar or the menu Peripherals – System Viewer.
- ❖ Note: The content of the menu Peripherals changes with the selected microcontroller.

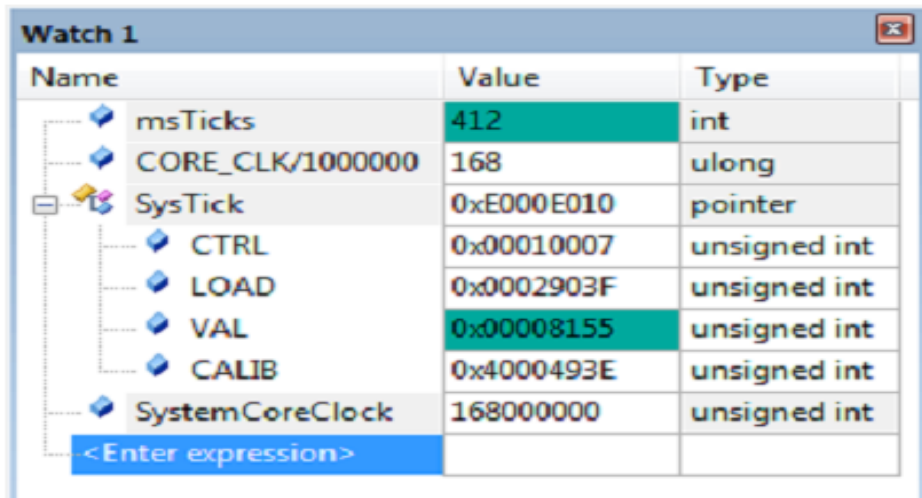


Property	Value
MODER	0x40000000
MODER15	0x01
OTYPER	0
GPIOB_OSPEEDR	0
PUPDR	0
IDR	0x00008020
IDR15	☒
ODR	0x00008000
BSRR	0
LCKR	0
AFRH	0

IDR
[Bits 31..0] RO (@ 0x40020C10) GPIO port input data register

Watch Window

- ❖ The Watch window allows you to observe program symbols, registers, memory areas, and expressions.
- ❖  Open a Watch window from the toolbar or the menu using View – Watch Windows.
- ❖ Add variables to the Watch window with:
 - Click on the field <Enter expression> and double-click or press F2.
 - In the Editor when the cursor is located on a variable, use the context menu select Add <item name> to...
 - Drag and drop a variable into a Watch window.
- ❖ The window content is updated when program execution is halted, or during program execution when View – Periodic Window Update is enabled.

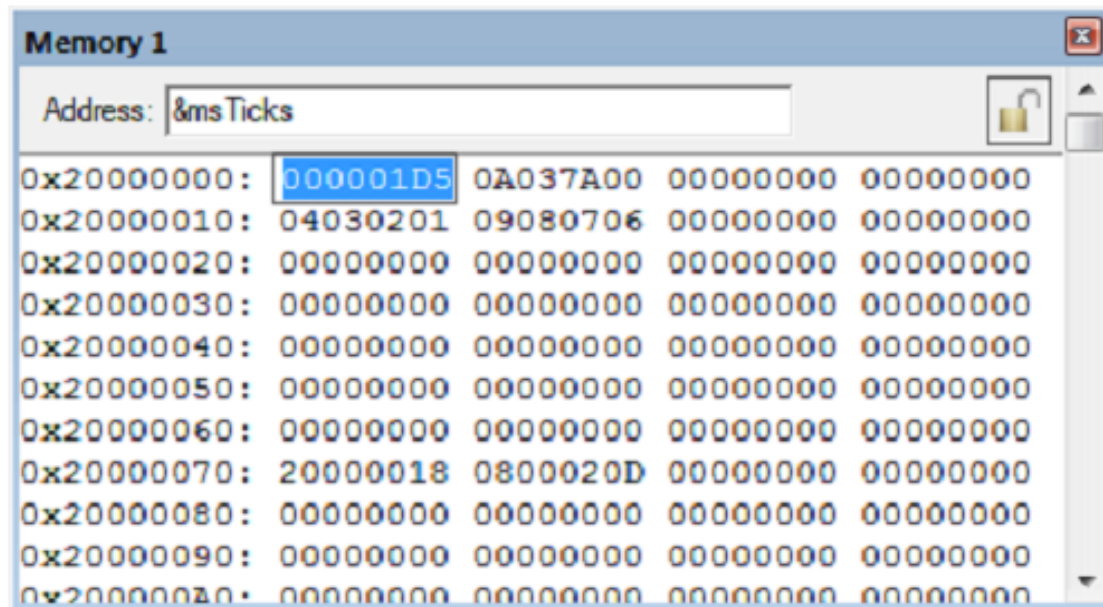


The screenshot shows a window titled "Watch 1" with a table of variables. The table has three columns: Name, Value, and Type. The variables listed are msTicks, CORE_CLK/1000000, SysTick, CTRL, LOAD, VAL, CALIB, and SystemCoreClock. The values are displayed in hexadecimal or decimal format. The Type column shows the data type for each variable. At the bottom of the table is a text field labeled "<Enter expression>".

Name	Value	Type
msTicks	412	int
CORE_CLK/1000000	168	ulong
SysTick	0xE000E010	pointer
CTRL	0x00010007	unsigned int
LOAD	0x0002903F	unsigned int
VAL	0x00008155	unsigned int
CALIB	0x4000493E	unsigned int
SystemCoreClock	168000000	unsigned int
<Enter expression>		

Memory Window

- ❖ Monitor memory areas using Memory Windows.
 - Enter an expression in the Address field to monitor the memory area.
 - To modify memory content, use the Modify Memory at ... command from context menu of the Memory window double-click on the value.
- ❖  Open a Memory window from the toolbar or the menu using View – Memory Windows.
- ❖  Stop refreshing the Memory window by clicking the Lock button. You can use the Lock feature to compare values of the same address space by viewing the same section in a second Memory window.



參考資料

The compiler

- ❖ The compiler, armcc, is an optimizing C and C++ compiler that compiles Standard C and Standard C++ source code into machine code for ARM architecture-based processors.
- ❖ The compiler compiles the following different standard of C and C++ source code
 - ISO C90 : The C standard and addenda was defined by the 1990
 - ISO C99 : The C standard and addenda was defined by the 1999
 - ISO C++03 : The C++ standard was defined by the 2003
 - ISO C++11 : The supported features of C++11 was defined by the 2011

呼叫 C 函式 in Assembly code

- ❖ The **IMPORT** directive provides the assembler with a name that is not defined in the current assembly.
- ❖ 在組合語言中呼叫C函式: specify the C function name with the IMPORT directive in assembly.

```
; int f(int i) { return g(i, 2*i, 3*i, 4*i, 5*i); }
PRESERVE8
EXPORT f
AREA f, CODE, READONLY
IMPORT g          ; i is in R0
STR lr, [sp, #-4]! ; preserve lr
ADD R1, R0, R0     ; compute 2*i (2nd param)
ADD R2, R1, R0     ; compute 3*i (3rd param)
ADD R3, R1, R2     ; compute 5*i
STR R3, [sp, #-4]! ; 5th param on stack
ADD R3, R1, R1     ; compute 4*i (4th param)
BL g              ; branch to C function
ADD sp, sp, #4     ; remove 5th param
LDR pc, [sp], #4   ; return
END
```

```
int g(int a, int b, int c, int d, int e)
{
    return a + b + c + d + e;
}
```

呼叫 assembly 函式 in C code

❖ 在C語言中呼叫組合語言函式:

- 在C語言中宣告該組合語言函式是 **EXTERN** directive.
- 在組合語言中輸出該函式. Use the **EXPORT** directive.

```
#include <stdio.h> extern void strcpy(char *d, const char *s);
int main()
{  const char *srcstr = "First string - source ";
   char dststr[] = "Second string - destination ";
   /* dststr is an array since we're going to change it */
   printf("Before copying:\n");
   printf("  %s\n  %s\n",srcstr,dststr);
   strcpy(dststr,srcstr);
   printf("After copying:\n");
   printf("  %s\n  %s\n",srcstr,dststr);
   return (0);
}
```

```
        PRESERVE8
        AREA      SCopy, CODE, READONLY
        EXPORT   strcpy

strcpy      ; R0 points to destination string.
            ; R1 points to source string.
        LDRB R2, [R1],#1 ; Load byte and update address.
        STRB R2, [R0],#1 ; Store byte and update address.
        CMP  R2, #0      ; Check for null terminator.
        BNE  strcpy      ; Keep going if not.
        BX   lr          ; Return.
        END
```

呼叫 C++ 函式 in C code

- ❖ 在C語言中呼叫C++函式: 在C++語言中宣告該函式是 **extern "C"**.
- ❖ 然後在C語言中宣告該C++語言函式是 **extern**.

C code

```
struct S {
    int i;
};
/* Declaration of the C++ function to be called from C */
extern void cppfunc(struct S *p);
int f(void) {
    struct S s;
    s.i = 2;                /* initialize 's' */
    cppfunc(&s);            /* call 'cppfunc' so it */
                           /* can change 's' */
    return s.i * 3;
}
```

C++ code

```
struct S {                // has no base classes or virtual functions
    S(int s) : i(s) { }
    int i;
};
extern "C" void cppfunc(S *p) {
    // Definition of the C++ function to be called from C.
    // The function is written in C++, only the linkage is C.
    p->i += 5;
}
```

呼叫 C 函式 in C++ code

- ❖ 在C++語言中呼叫C函式: 在C++語言中宣告該C語言函式是 **extern "C"**.

C++ code

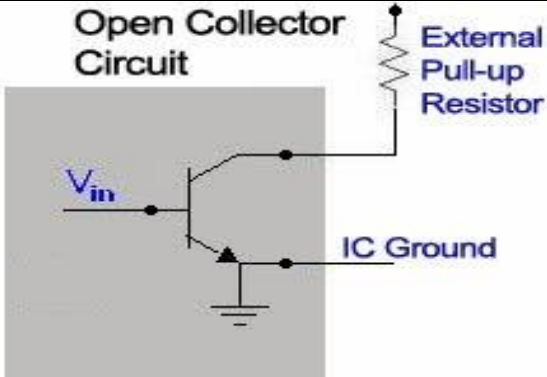
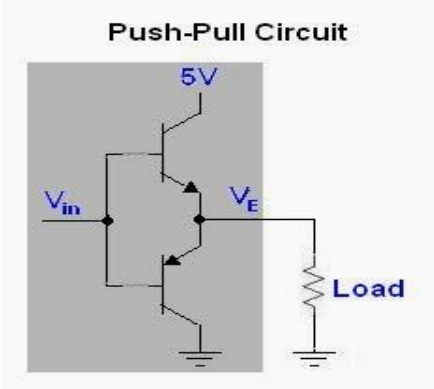
```
struct S {                // has no base classes
                        // or virtual functions
    S(int s) : i(s) { }
    int i;
};
// declare the C function to be called from C++
extern "C" void cfunc(S *);
int f(){
    S s(2);                // initialize 's'
    cfunc(&s);              // call 'cfunc' so it can change 's'
    return s.i * 3;
}
```

C code

```
struct S {
    int i;
};
void cfunc(struct S *p) {
    /* the definition of the C function to be called from C++ */
    p->i += 5;
}
```

Open-drain & Push-pull的電路比較

- ❖ Open-Drain：看到Open就想到輸出是開路的，所以使用時要加上拉電阻。Open-drain的電路架構使用一個N-MOS(or BJT-NPN)當開關，所以輸出的電壓必需由外部決定，利用外部電路的驅動能力，減少IC內部的驅動，或驅動比晶片電源電壓高的負載。
- ❖ Push-Pull：又推又拉的它都做了，外部就不用再多加什麼元件。其實Push-Pull的內部使用了兩個電晶體當Totem Pole，當輸出高電位時，上面的電晶體導通、下面的電晶體關閉，使輸出為高電位；相反的，輸出低電位時，上面的電晶體關閉，下面的電晶體導通，使輸出為低電位。所以輸出不需要額外加上拉電阻。

名稱	Open-drain (MOSFET) or open collector (BJT)	Push-Pull
電路架構	 Open Collector Circuit External Pull-up Resistor IC Ground V_{in}	 Push-Pull Circuit 5V V_{in} V_E Load
輸出	輸出要加上拉電阻，否則是浮接。	輸出可為 Hi 或 Low 準位。
特點	1.可做電壓轉換-Level shift IC 內部僅需很小的閘極驅動電流	1.可以吸電流。 2.可以灌電流。 3.輸出電壓由 IC 電源決定。